

Aprendizaje supervisado: Clasificación

Machine Learning

SUPPORT VECTOR MACHINE



Flavio E. Spetale

2025

Bayes Ingenuo

El algoritmo de clasificación de Bayes Ingenuo se basa en el concepto de probabilidad condicional y busca maximizar la verosimilitud del modelo, i.e., otorgar mayor importancia a aquellos eventos que son realmente relevantes en el juego de datos.

Se supone que los predictores en un modelo Bayes Ingenuo son condicionalmente independientes o no están relacionados con ninguna de las otras características del modelo.

También supone que todas las características contribuyen por igual al resultado. Si bien estos supuestos a menudo se violan en escenarios del mundo real, simplifica un problema de clasificación al hacerlo más manejable computacionalmente.

Por tanto, sólo se requerirá una probabilidad única para cada variable, facilitando el cálculo del modelo. A pesar de esta suposición poco realista de independencia, el algoritmo de clasificación funciona bien, particularmente con tamaños de muestra pequeños.

Bayes Ingenuo

En general, los métodos estadísticos suelen estimar un conjunto de parámetros probabilísticos, que expresan la probabilidad condicionada de cada clase dadas las propiedades de un ejemplo (descrito en forma de atributos).

Un aspecto central en el algoritmo de Bayes Ingenuo es el concepto de probabilidad condicionada.

Probabilidad condicional

La probabilidad condicional $P(A|B)$ es la probabilidad de que ocurra un evento A, sabiendo que también sucede otro evento B

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Bayes Ingenuo

	Nombre	Tipo	Ataque	Defensa	Velocidad	HP
	Growlithe	Fuego	Bajo	Débil	Baja	Baja
	Charizard	Fuego	Medio	Media	Alta	Media
	Arcanine	Fuego	Alto	Media	Alta	Alta
	Pyroar	Fuego	Bajo	Débil	Alta	Media
	Entei	Fuego	Alto	Dura	Alta	Alta
	Houndoom	Siniestro	Medio	Débil	Alta	Media
	Mightyena	Siniestro	Medio	Media	Baja	Media
	Hydreigon	Siniestro	Alto	Dura	Alta	Alta
	Thievul	Siniestro	Bajo	Débil	Alta	Baja
	Obstagoon	Siniestro	Medio	Dura	Alta	Alta
	Darkrai	Siniestro	Medio	Dura	Alta	Baja
	Bulbasaur	Planta	Bajo	Débil	Baja	Baja
	Torterra	Planta	Alto	Dura	Baja	Alta

Bayes Ingenuo

Probabilidades:

$$p(\text{tipo} = \text{fuego}) = \frac{5}{13} = 0.385$$

$$p(\text{tipo} = \text{siniestro}) = \frac{6}{13} = 0.462$$

$$p(\text{tipo} = \text{planta}) = \frac{2}{13} = 0.153$$

¿Cuántos tienen un ataque alto dado que sea del tipo siniestro?

Observamos que de entre las 6 clasificadas como siniestro, solo uno tiene un ataque alto.

$$p(\text{alto}|\text{siniestro}) = \frac{p(\text{alto} \cap \text{siniestro})}{p(\text{siniestro})} = \frac{1/13}{6/13} = \frac{1}{6} = 0.166$$

Algoritmo Bayes Ingenuo

El algoritmo Bayes Ingenuo clasifica nuevos ejemplos $d = (d_1, \dots, d_m)$ asignándole la clase c que maximiza la probabilidad condicional de la clase, dada la secuencia observada de atributos del ejemplo.

$$\arg \max_c P(c|d_1, \dots, d_m) = \arg \max_c \frac{P(d_1, \dots, d_m|c)P(c)}{P(d_1, \dots, d_m)} \approx \arg \max_c P(c) \prod_{i=1}^m P(d_i|c)$$

donde $P(c)$ y $P(d_i|c)$ se estiman a partir del conjunto de entrenamiento, utilizando las frecuencias relativas (estimación de la máxima verosimilitud, en inglés maximum likelihood estimation).

En el caso de tener un conjunto de datos que incluya *atributos numéricos continuos* es necesario algún tipo de preproceso antes de poder aplicar el método descrito.

Algoritmo Bayes Ingenuo

Existen dos alternativas principales para lidiar con los atributos continuos:

- La forma más simple consiste en categorizar los atributos continuos, de forma que sean convertidos en intervalos discretos y se puedan tratar de igual forma que los atributos nominales.
- Otra opción consiste en asumir que los valores de cada clase siguen una distribución gaussiana, y aplicar la siguiente fórmula:

$$p(d = v|c) = \frac{1}{\sqrt{2\pi\sigma_c^2}} e^{\left(-\frac{(v-\mu_c)^2}{2\sigma_c^2}\right)}$$

donde d corresponde al atributo, v a su valor, c a la clase, μ_c al promedio de valores de la clase c , y σ_c^2 a su desviación estándar.

En el caso que dentro del conjunto de test aparezca una pareja $\langle \text{atributo}, \text{valor} \rangle$ que no haya aparecido en el conjunto de entrenamiento, no dispondremos del valor $p(d_i|c)$.

Para solucionar este problema se suele aplicar alguna técnica de suavizado, como por ejemplo asignar como probabilidad condicional el valor $\frac{p(c)}{n}$, donde n indica el número de instancias de entrenamiento.

Ejemplo de Bayes Ingenuo

Probabilidades condicionales organizadas por *<atributo, valor>*.

Atributo - Valor	Fuego	Siniestro	Planta
Ataque - Bajo	2/5	4/6	1/2
Ataque - Medio	1/5	1/6	0
Ataque - Alto	2/5	1/6	1/2
Defensa - Débil	2/5	2/6	1/2
Defensa - Media	2/5	1/6	0
Defensa - Dura	1/5	3/6	1/2
Velocidad - Baja	1/5	1/6	0
Velocidad - Alta	4/5	5/6	1
HP - Baja	1/5	2/6	1/2
HP - Media	2/5	2/6	0
HP - Alta	2/5	2/6	1/2

Ejemplo de Bayes Ingenuo

Con esto habremos terminado el proceso de entrenamiento.

Ahora trataremos de clasificar una entrada nueva donde los atributos son Ataque: Alto, Defensa: Media, Velocidad: Alta y HP: Media. Para ello aplicamos el criterio de Bayes Ingenuo:

$$\arg \max_c P(c) \prod_{i=1}^m P(d_i|c)$$

	Tipo	Ataque $p(d_i c)$	Defensa $p(d_i c)$	Velocidad $p(d_i c)$	HP $p(d_i c)$	
$p(c)$	?	Alto	Media	Alta	Media	
5/13	Fuego	2/5	2/5	4/5	2/5	$\frac{5}{13} * \left(\frac{2}{5} * \frac{2}{5} * \frac{4}{5} * \frac{2}{5}\right) = \frac{160}{8125} = 0.0196$
6/13	Siniestro	1/6	1/6	5/6	2/6	$\frac{6}{13} * \left(\frac{1}{6} * \frac{1}{6} * \frac{5}{6} * \frac{2}{6}\right) = \frac{60}{16848} = 0.0036$
2/13	Planta	1/2	0	1	0	$\frac{2}{13} * \left(\frac{1}{2} * \frac{0}{2} * \frac{2}{2} * \frac{0}{2}\right) = \frac{0}{208} = 0.0000$

Ejemplo de Bayes Ingenuo

Clasificaremos el nuevo dato con clase «fuego», dado que presenta la mayor verosimilitud de acuerdo con Bayes Ingenuo.



Infernape

Tipos de clasificadores de Bayes Ingenuo

- Gaussian Naïve Bayes (GaussianNB): Es una variante del clasificador Bayes Ingenuo, que se utiliza con distribuciones gaussianas, i.e., distribuciones normales y variables continuas. Este modelo se ajusta encontrando la media y la desviación estándar de cada clase.
- Multinomial Naïve Bayes (MultinomialNB): Supone que las características provienen de distribuciones multinomiales. Esta variante es útil cuando se utilizan datos discretos, como recuentos de frecuencia, y normalmente se aplica en casos de uso de procesamiento de lenguaje natural.
- Bernoulli Naïve Bayes (BernoulliNB): Es otra variante del clasificador Bayes Ingenuo, que se usa con variables booleanas, i.e., variables con dos valores (1 y 0).

Todo esto se puede implementar a través de la biblioteca Python de Scikit Learn.

Bayes Ingenuo

Ventajas:

1. Menos complejo: en comparación con otros clasificadores, Bayes Ingenuo se considera un clasificador más simple ya que los parámetros son más fáciles de estimar.
2. Se escala bien: en comparación con la regresión logística, Bayes Ingenuo se considera un clasificador rápido y eficiente que es bastante preciso cuando se cumple el supuesto de independencia condicional. También tiene bajos requisitos de almacenamiento.
3. Puede manejar datos de alta dimensión: la clasificación de documentos puede tener una gran cantidad de dimensiones, lo que puede resultar difícil de gestionar para otros clasificadores.

Desventajas:

1. Supuesto central poco realista: si bien el supuesto de independencia condicional en general funciona bien, no siempre se cumple, lo que lleva a clasificaciones incorrectas.
2. Sujeto a frecuencia cero: La frecuencia cero ocurre cuando una variable categórica no existe dentro del conjunto de entrenamiento. La probabilidad en este caso sería cero, y dado que este clasificador multiplica todas las probabilidades condicionales juntas, esto también significa que la probabilidad posterior será cero. Para evitar este problema, se puede aprovechar el suavizado de Laplace.

Ejemplo en python

Conjunto de datos: *Clasificación de frutas*

Información del conjunto de datos:

El conjunto de datos esta formado por xxx frutas, 70 elementos cada una.
Todos estos parámetros fueron de valor real continuo.

Información de atributos:

Para construir los datos, se midieron 34 parámetros fenotípicos y geométricos de las frutas.

Ejemplo en python

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics

dataFruit = pd.read_csv("FruitBD.csv")
xFruit = dataFruit.iloc[:, [2,8,15,23]].values
yFruit = dataFruit['Clase']

xFruit = StandardScaler().fit_transform(xFruit)
xFruitTrain, xFruitTest, yFruitTrain, yFruitReal = train_test_split(xFruit, yFruit, test_size=0.2)
```

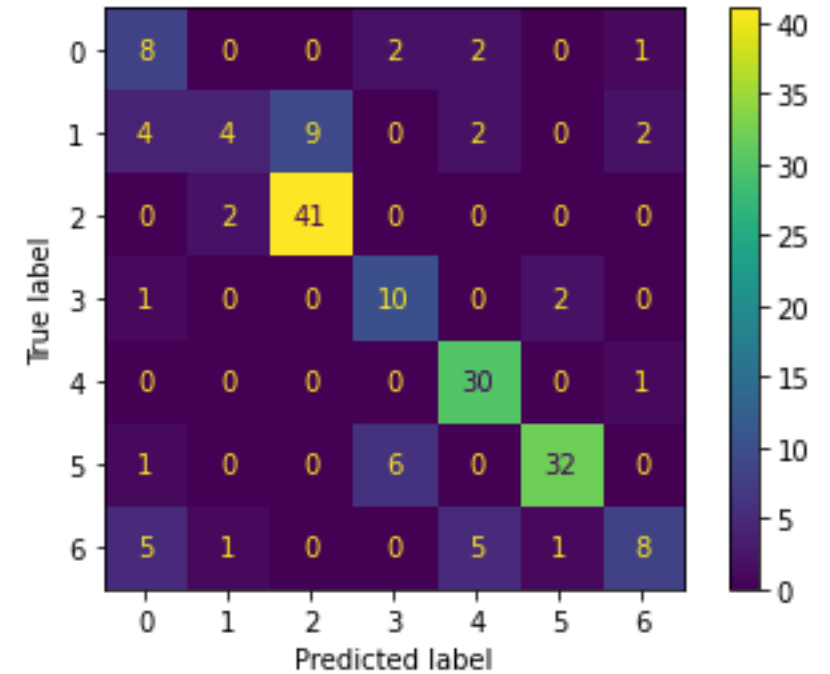
Ejemplo en python

""" Naive Bayes - Classification """

```
modelNB = GaussianNB()  
modelNB.fit(xFruitTrain, yFruitTrain)
```

```
yFruitPred = modelNB.predict(xFruitTest)
```

```
cm = metrics.confusion_matrix(yFruitReal, yFruitPred)  
metrics.ConfusionMatrixDisplay(confusion_matrix=cm).plot()
```



metrics.precision_score(yFruitReal, yFruitPred, average='macro') → 0.674

metrics.recall_score(yFruitReal, yFruitPred, average='macro') → 0.673

metrics.accuracy_score(yFruitReal, yFruitPred) → 0.738

Ejemplo en python

""" Naive Bayes - All features """

```
xFruit = dataFruit.drop('Clase', axis=1)
xFruit = StandardScaler().fit_transform(xFruit)
xFruitTrain, xFruitTest, yFruitTrain, yFruitReal = train_test_split(xFruit, yFruit,
test_size=0.2)
```

```
modelNB = GaussianNB()
modelNB.fit(xFruitTrain, yFruitTrain)
```

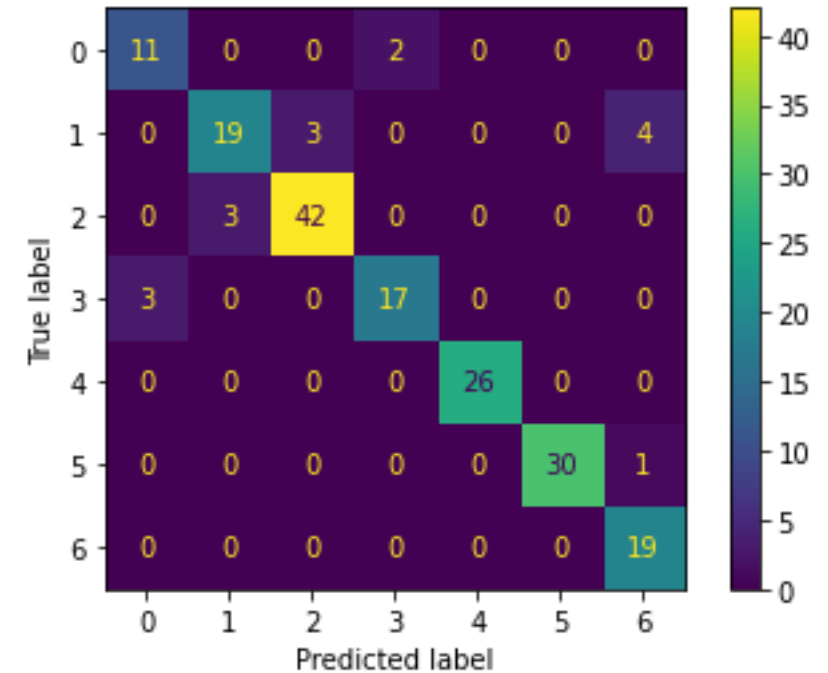
```
yFruitPred = modelNB.predict(xFruitTest)
```

```
cm = metrics.confusion_matrix(yFruitReal, yFruitPred)
metrics.ConfusionMatrixDisplay(confusion_matrix=cm).plot()
```

metrics.precision_score(yFruitReal, yFruitPred, average='macro') → 0.895

metrics.recall_score(yFruitReal, yFruitPred, average='macro') → 0.903

metrics.accuracy_score(yFruitReal, yFruitPred) → 0.911



k vecinos más cercanos (k-NN)

El algoritmo k-NN, es un clasificador de aprendizaje supervisado no paramétrico (no hace suposiciones sobre los datos subyacentes), ***no genera un modelo*** fruto del aprendizaje con datos de entrenamiento, sino que el aprendizaje sucede en el mismo momento en el que se pide clasificar una nueva instancia.

A este tipo de algoritmos se los denomina métodos de aprendizaje perezoso (lazy learning methods).

Todo el cálculo ocurre cuando se realiza una clasificación o predicción. Dado que depende en gran medida de la memoria para almacenar todos sus datos de entrenamiento, también se lo denomina método de aprendizaje basado en instancias o basado en la memoria.

Si bien no es tan popular hoy en día, sigue siendo uno de los primeros algoritmos que uno aprende en la ciencia de datos debido a su simplicidad y precisión.

A medida que crece un conjunto de datos, k-NN se vuelve cada vez más ineficiente, lo que compromete el rendimiento general del modelo.

El funcionamiento del algoritmo es muy simple. Para cada nueva instancia a clasificar, se calcula la distancia con todas las instancias de entrenamiento, se seleccionan las k instancias más cercanas y su clase se determinará como la clase mayoritaria de sus k instancias más cercanas.

k-NN: métricas de distancia

La métrica de similitud utilizada debería tener en cuenta la importancia relativa de cada atributo, puesto que esta influirá fuertemente en la relaciones de cercanía que se irán estableciendo en el proceso de construcción del algoritmo.

La métrica de distancia puede llegar a contener pesos que nos ayudarán a calibrar el algoritmo de clasificación, convirtiéndola de hecho en una métrica personalizada.

Debería ser eficiente computacionalmente, ya que deberemos ejecutar el cálculo de similitud muchas veces durante el proceso de clasificación de nuevas instancias.

Distancia euclidiana (p=2): Esta es la medida de distancia más utilizada y está limitada a vectores de valor real.

$$d(x, y) = \sqrt{\sum_{i=1}^n (y_i - x_i)^2}$$

k-NN: métricas de distancia

Distancia Manhattan (p=1): Esta es también otra métrica de distancia popular, que mide el valor absoluto entre dos puntos.

$$d(x, y) = \sum_{i=1}^n |y_i - x_i|$$

Distancia Minkowski: Esta distancia puede considerarse una generalización de las distancias euclideas y Manhattan

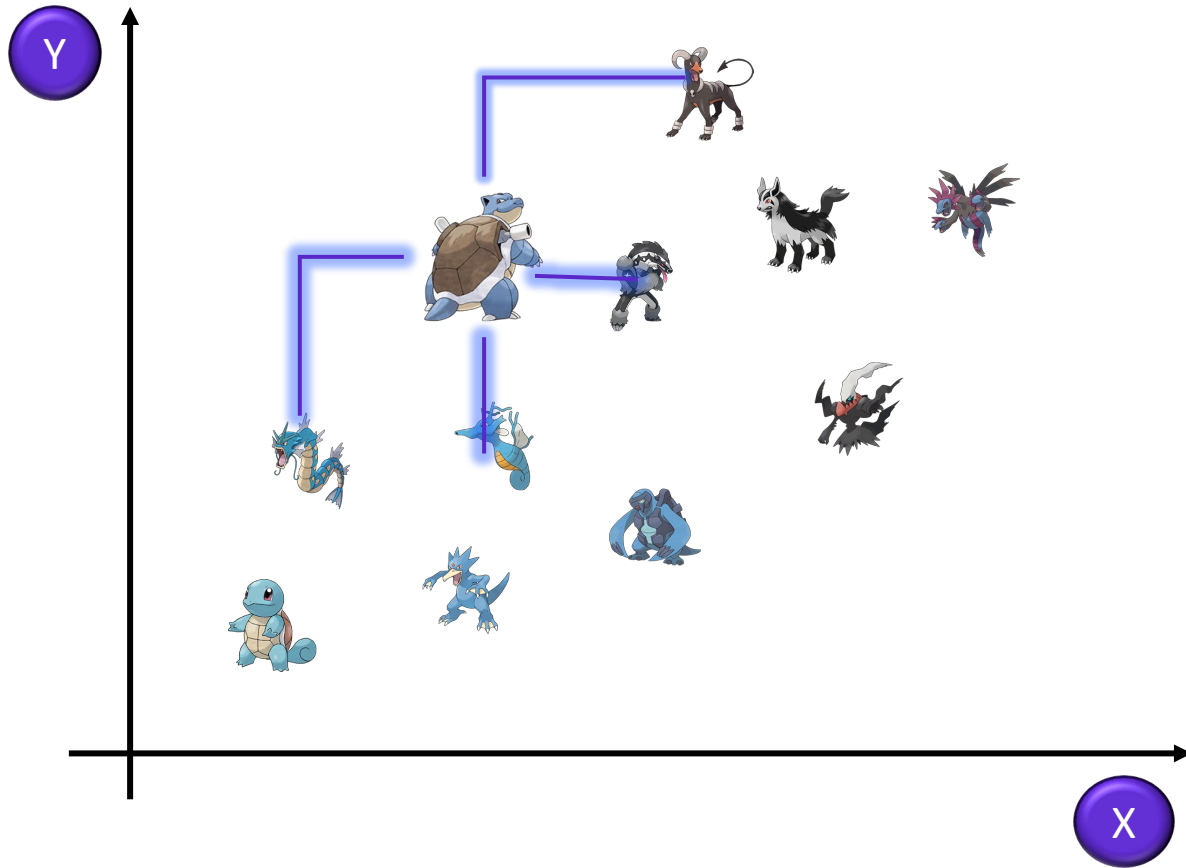
$$d(x, y) = \left(\sum_{i=1}^n |y_i - x_i|^p \right)^{1/p}$$

Distancia de Hamming: Esta técnica se usa típicamente con vectores booleanos o de cadena, identificando los puntos donde los vectores no coinciden.

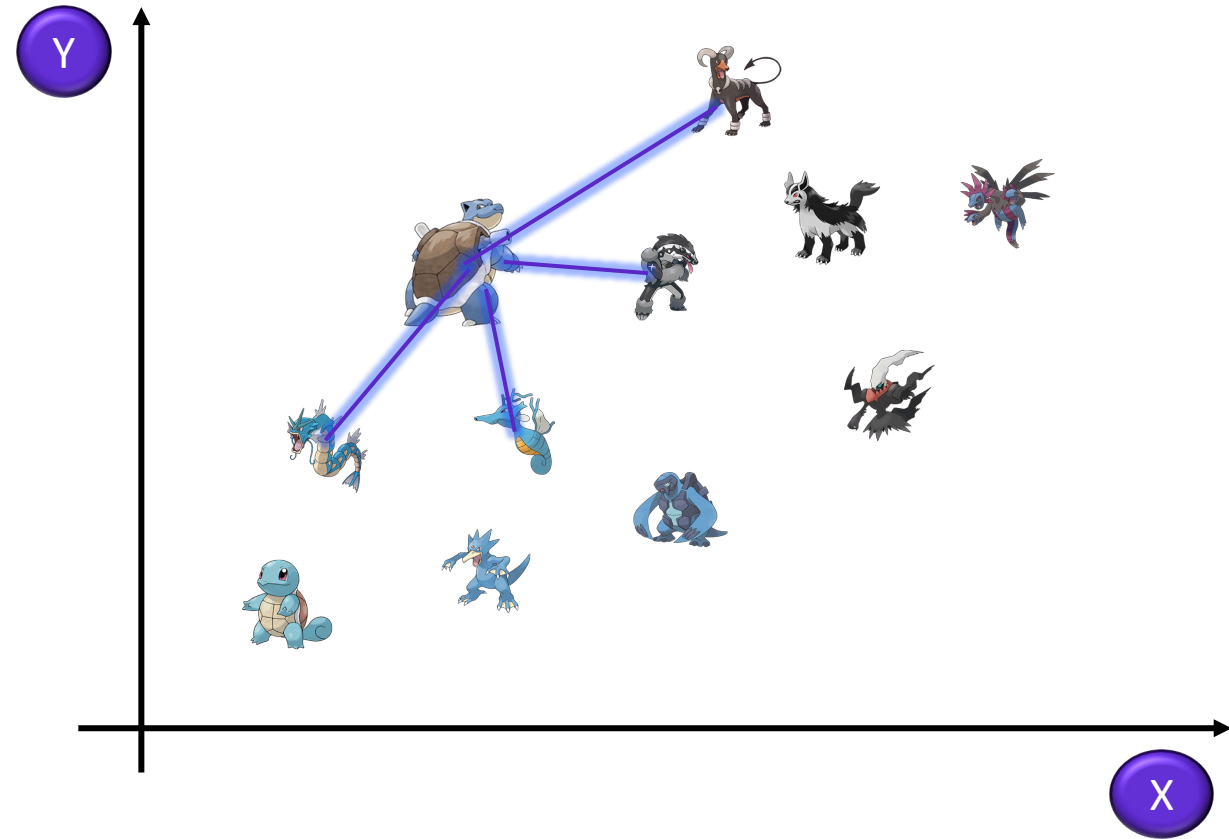
$$d(x, y) = \sum_{i=1}^n |y_i - x_i|$$

$$\text{si } x_i = y_i \rightarrow 0 \text{ sino } x_i \neq y_i \rightarrow 1$$

Métricas de distancia



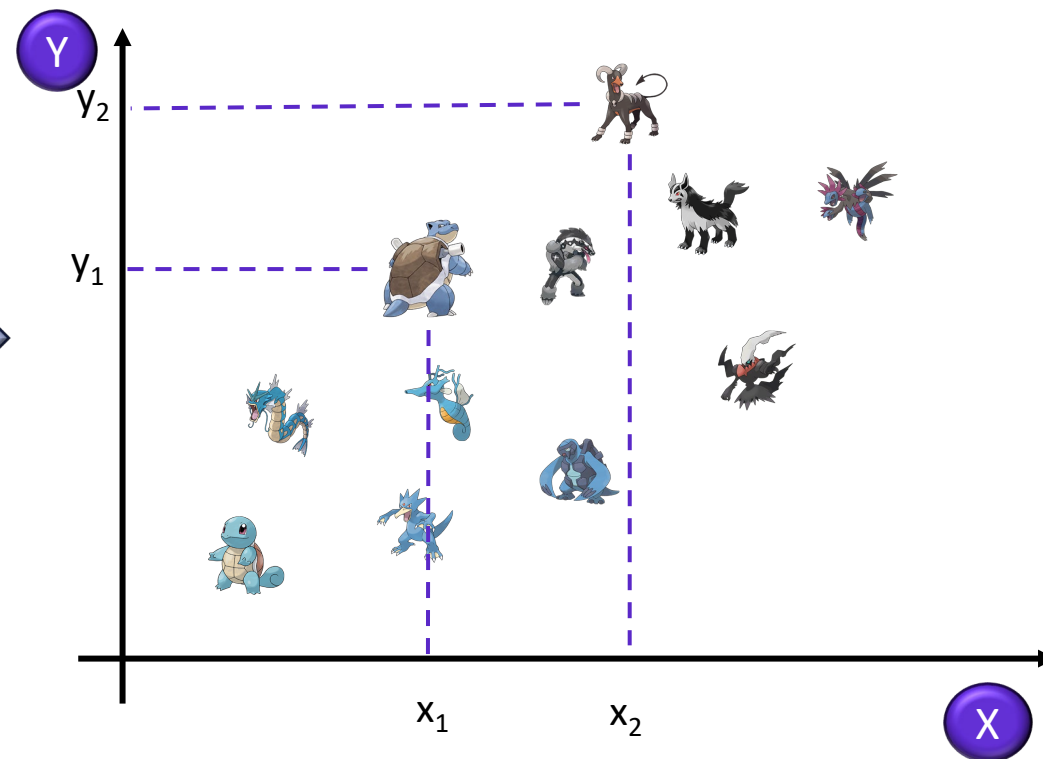
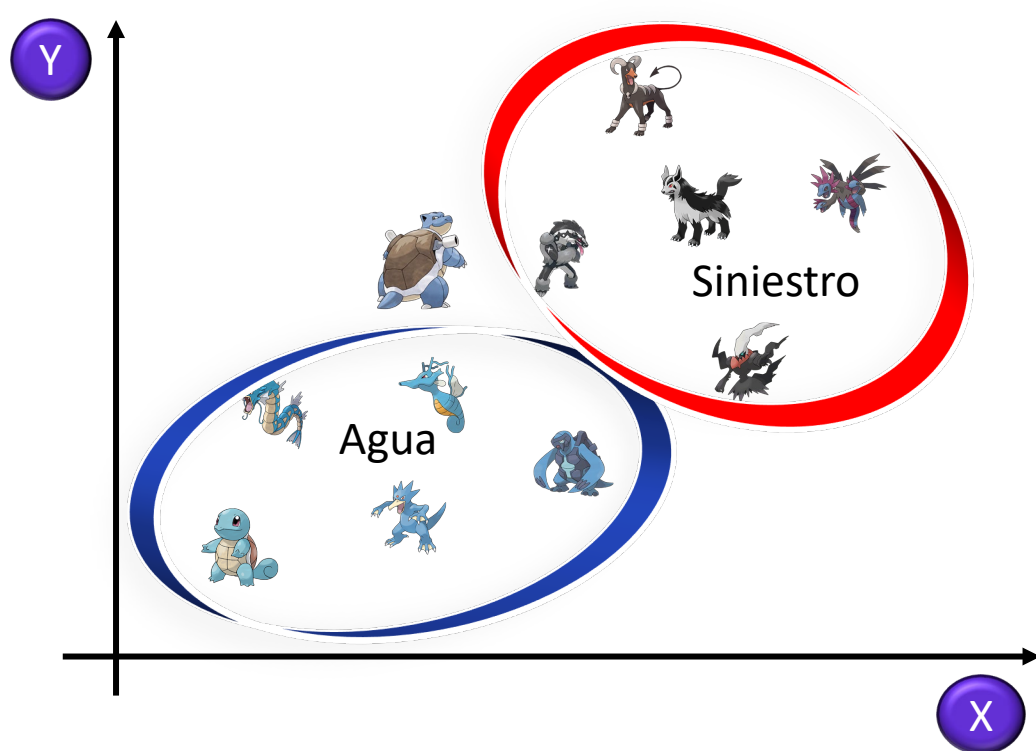
Distancia Manhattan



Distancia euclidiana

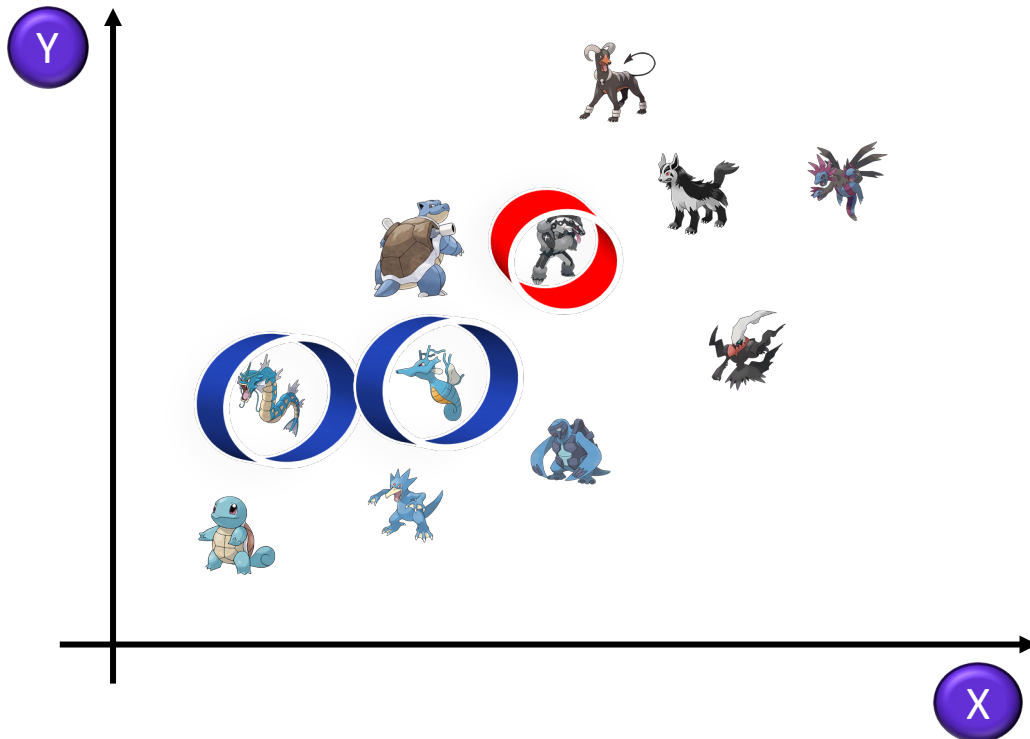
Algoritmo k-NN

1. Calcular la distancia de este nuevo punto hacia todos los otros puntos que ya se encuentran etiquetados con su clase correspondiente para después ordenar de menor a mayor.
2. Con las distancias ordenadas de menor a mayor basta con tomar los mas cercanos al punto a clasificar, lo cual esta determinado con el valor K seleccionado. Aquí consideramos $k=3$.



Algoritmo k-NN

3. Utilizamos un método de agregación, voto mayoritaria, para estimar a la categoría que pertenece el nuevo dato.



Clasificamos el nuevo dato
con clase «Agua»



Blastoise

k-NN: Selección del valor k

Optimizar el valor de k es de suma importancia, ya que al elegir un valor de k muy pequeño es posible que el ruido de los datos tome importancia, por otro lado si el valor de k es alto la clase que sea mayoría tendrá un peso importante.

Este valor suele fijarse tras un proceso de pruebas con varias instancias.

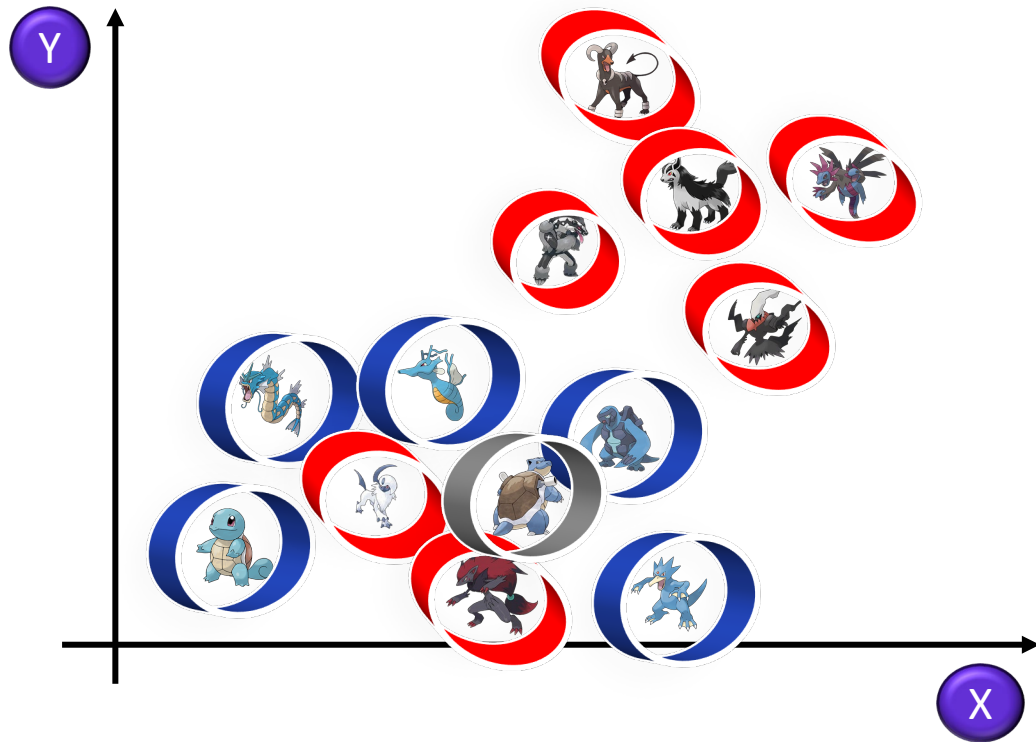
Se suele escoger un número impar o primo para minimizar la posibilidad de empates en el momento de decidir la clase de una nueva instancia.

Aun así, cuando se produce un empate se debe decidir cómo clasificar la instancia. Algunas alternativas pueden ser, por ejemplo, no dar predicción o dar la clase más frecuente en el conjunto de aprendizaje de las clases que han generado el empate.

Otro factor importante que se debe considerar antes de aplicar el algoritmo k-NN es el rango u orden de los datos. Es importante expresar los distintos atributos en valores que sean «comparables», i.e., normalizarlos.

k-NN: Selección del valor k

k pequeño (*overfitting*)



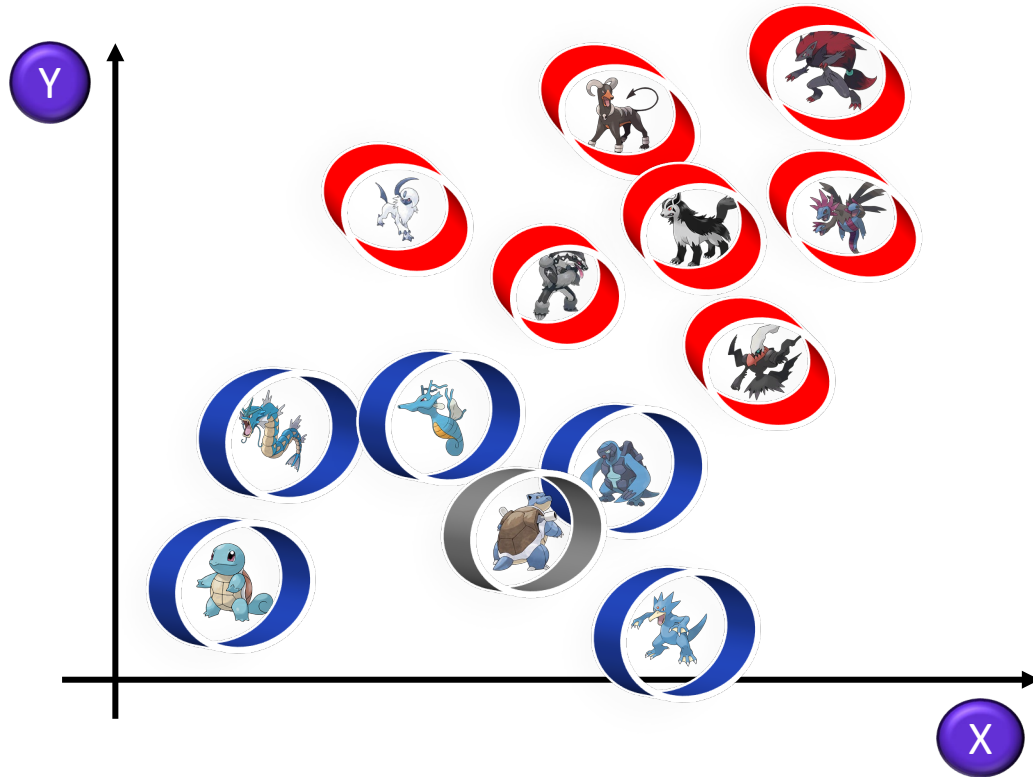
Clasificamos el nuevo dato
con clase «Siniestro»



Tomando un valor de $k = 3$ el nuevo dato será etiquetado como Siniestro aunque visualmente es evidente que pertenece a Agua, sin embargo aquí es donde el **ruido toma importancia**, y se dice que el modelo está sobreajustado.

k-NN: Selección del valor k

k alto (underfitting)



Clasificamos el nuevo dato
con clase «Siniestro»



Tomando un valor de $k = 12$ el nuevo dato será etiquetado como Siniestro, subajuste del algoritmo, ya que visualmente el nuevo dato pertenece a la categoría Agua, sin embargo al ser la clase Siniestro mayoría, es aquí donde será clasificado.

k-NN: Selección del valor k

Al analizar los extremos (k muy pequeño y k muy alto) es clara la importancia de un k-valor adecuado, normalmente se escoge como un *buen valor de inicio la raíz cuadrada del número de datos para el entrenamiento*, sin embargo es buena practica tomar varios valores de k, y observar cual de estos es el que nos da una mejor clasificación de los datos de entrenamiento.

Ejemplo en python

```
KNeighborsClassifier(  
    n_neighbors=5,      # Número de vecinos a considerar  
    weights='uniform',  # Cómo ponderar distancias  
    algorithm='auto',   # Algoritmo para calcular los vecinos  
    leaf_size=30,       # El tamaño de la hoja para agilizar las búsquedas  
    p=2,                # El parámetro de potencia para la métrica de Minkowski  
    metric='minkowski', # El tipo de distancia a utilizar  
    n_jobs=None         # Trabajos en paralelos a ejecutar  
)
```

Ejemplo en python

Codificación one-hot de variables categóricas en Sklearn

```
from sklearn.preprocessing import OneHotEncoder  
from sklearn.compose import make_column_transformer
```

```
X = df.drop(columns = ['clases'])  
y = df ['clases']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state = 100)
```

```
column_transformer = make_column_transformer( (OneHotEncoder(), ['x1', 'x2', 'x3']),  
remainder='passthrough')
```

```
X_train = column_transformer.fit_transform(X_train)  
X_train = pd.DataFrame(data=X_train, columns=column_transformer.get_feature_names())
```

Ejemplo en python

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics

dataFruit = pd.read_csv("FruitBD.csv")

yFruit = dataFruit['Clase']

xFruit = dataFruit.drop('Clase', axis=1)
xFruit = StandardScaler().fit_transform(xFruit)
xFruitTrain, xFruitTest, yFruitTrain, yFruitReal = train_test_split(xFruit, yFruit, test_size=0.2)

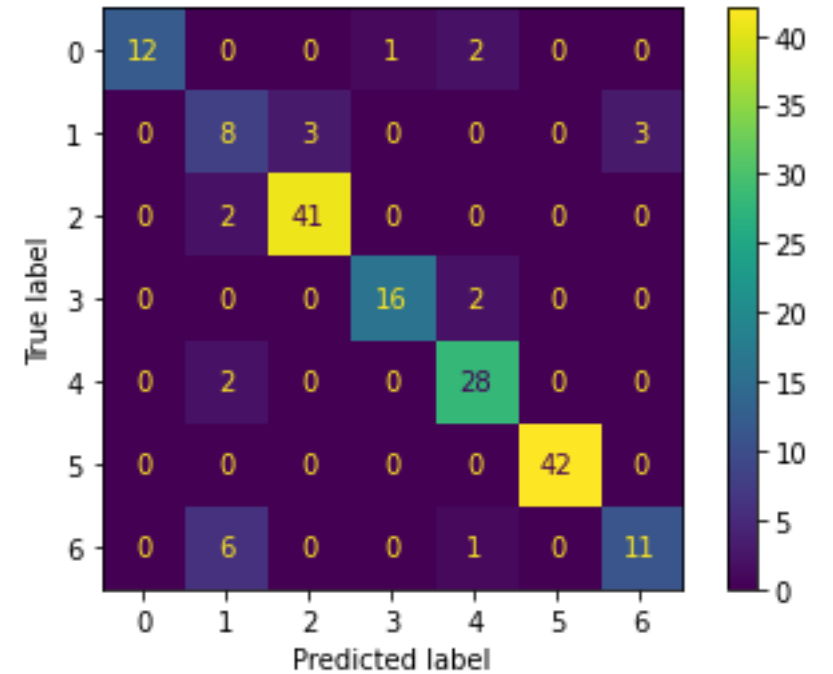
modelkNN = KNeighborsClassifier(n_neighbors=3, metric='minkowski', p=2)

modelkNN.fit(xFruitTrain, yFruitTrain)

yFruitPred = modelkNN.predict(xFruitTest)
```

Ejemplo en python

```
cm = metrics.confusion_matrix(yFruitReal, yFruitPred)
metrics.ConfusionMatrixDisplay(confusion_matrix=cm).plot()
```



`metrics.precision_score(yFruitReal, yFruitPred, average='macro')` → 0.850

`metrics.recall_score(yFruitReal, yFruitPred, average='macro')` → 0.822

`metrics.accuracy_score(yFruitReal, yFruitPred)` → 0.877

Ejemplo en python

```
from sklearn.model_selection import GridSearchCV
```

```
params = {  
    'n_neighbors': range(1, 15, 2),  
    'p': [1,2],  
    'weights': ['uniform', 'distance']  
}
```

```
clf = GridSearchCV( estimator=KNeighborsClassifier(), param_grid=params, cv=5, n_jobs=5,verbose=1)
```

```
clf.fit(xFruitTrain, yFruitTrain)
```

```
print(clf.best_params_)      →      {'n_neighbors': 5, 'p': 1, 'weights': 'distance'}
```

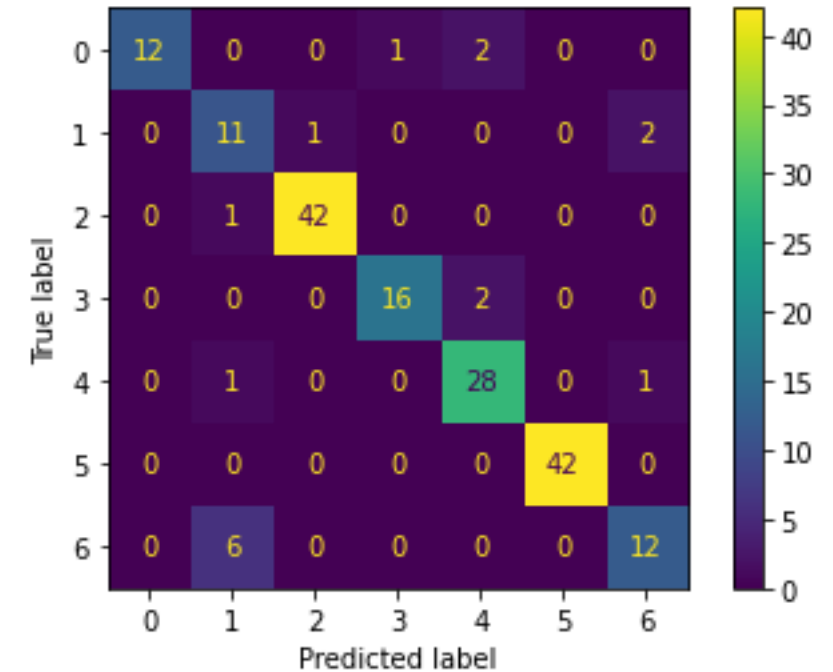
```
modelkNN = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=1, weights='distance')
```

```
modelkNN.fit(xFruitTrain, yFruitTrain)
```

```
yFruitPred = modelkNN.predict(xFruitTest)
```

Ejemplo en python

```
cm = metrics.confusion_matrix(yFruitReal, yFruitPred)
metrics.ConfusionMatrixDisplay(confusion_matrix=cm).plot()
```



`metrics.precision_score(yFruitReal, yFruitPred, average='macro')` → 0.881

`metrics.recall_score(yFruitReal, yFruitPred, average='macro')` → 0.864

`metrics.accuracy_score(yFruitReal, yFruitPred)` → 0.905

Máquinas de soporte vectorial (SVM)

La gran aportación de Vapnik radica en que construye un método que tiene por objetivo producir predicciones en las que se puede tener mucha confianza, en lugar de lo que se ha hecho tradicionalmente, que consiste en construir hipótesis que cometan pocos errores.

La hipótesis tradicional se basa en lo que se conoce como minimización del riesgo empírico (empirical risk minimization) mientras que el enfoque de las SVM se basa en la minimización del riesgo estructural (structural risk minimization), de modo que lo que se busca es construir modelos que estructuralmente tengan poco riesgo de cometer errores ante clasificaciones futuras.

El concepto de minimización del riesgo estructural fue introducido en 1974 por Vapnik y Chervonenkis.

La idea detrás de las máquinas de soporte vectorial es muy intuitiva. Si la frontera definida entre dos regiones con elementos de clases diferentes es compleja, en lugar de construir un clasificador complejo que reproduzca dicha frontera, lo que se intenta es «doblar» el espacio de datos en un espacio de mayor dimensionalidad de forma que con un único corte se puedan separar fácilmente ambas regiones.

Tipos de algoritmos SVM

SVM lineal: Solo cuando los datos son perfectamente separables linealmente.

Perfectamente separable linealmente significa que los puntos de datos se pueden clasificar en dos clases utilizando una sola línea recta (si son 2D).

SVM no lineal: Cuando los datos no son linealmente separables.

Esto ocurre cuando los puntos de datos no se pueden separar en dos clases utilizando una línea recta (si son 2D).

En estos casos, utilizamos técnicas avanzadas como trucos de kernel para clasificarlos.

En la mayoría de las aplicaciones del mundo real, no encontramos puntos de datos linealmente separables, por lo que utilizamos trucos de kernel para resolverlos.

Características de SVM

1. **Maximiza el margen:** Se enfoca en encontrar el límite de decisión que maximiza el margen (la distancia entre el límite y los puntos de datos más cercanos de cada clase). Esto lo hace más robusto ante nuevos datos.
2. **Funciona con datos no lineales:** Puede funcionar datos no lineales mediante el "truco del kernel", que transforma los datos en un espacio de mayor dimensión donde es más fácil separarlos.
3. **Eficaz en grandes dimensiones:** Funciona bien incluso cuando el número de características (dimensiones) es mucho mayor que el número de muestras, lo que lo hace adecuado para conjuntos de datos complejos.
4. **Resistente al sobreajuste:** Al centrarse en los puntos más cercanos al límite (vectores de soporte), SVM tiene menos probabilidades de sobreajustarse, especialmente en conjuntos de datos más pequeños.
5. **Requiere ajuste:** Requiere un ajuste cuidadoso de los parámetros (como la elección del kernel y la regularización) para lograr un rendimiento óptimo, lo que puede requerir mucho tiempo.

¿Cómo funciona el algoritmo de SVM?

El SVM se define únicamente en términos de los vectores de soporte; no tenemos que preocuparnos por otras observaciones, ya que el margen se crea utilizando los puntos más cercanos al hiperplano (vectores de soporte).

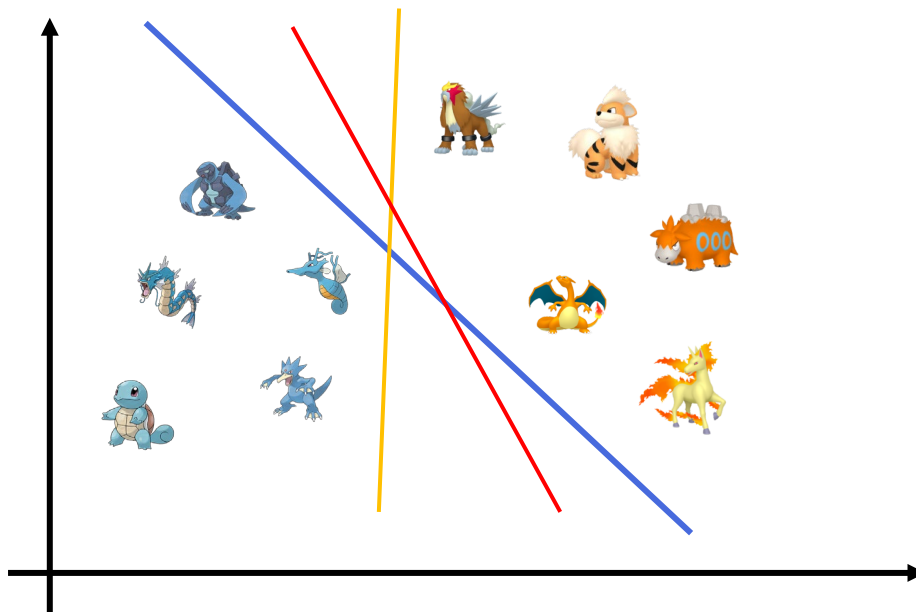
Supongamos que tenemos un conjunto de datos con dos clases (agua y fuego). Queremos clasificar el nuevo dato (observación) como agua o fuego.



SVM: Hiperplano separador

Para clasificar estos puntos, podemos tener varios límites de decisión, pero la pregunta es ¿cuál es el mejor y cómo lo encontramos?

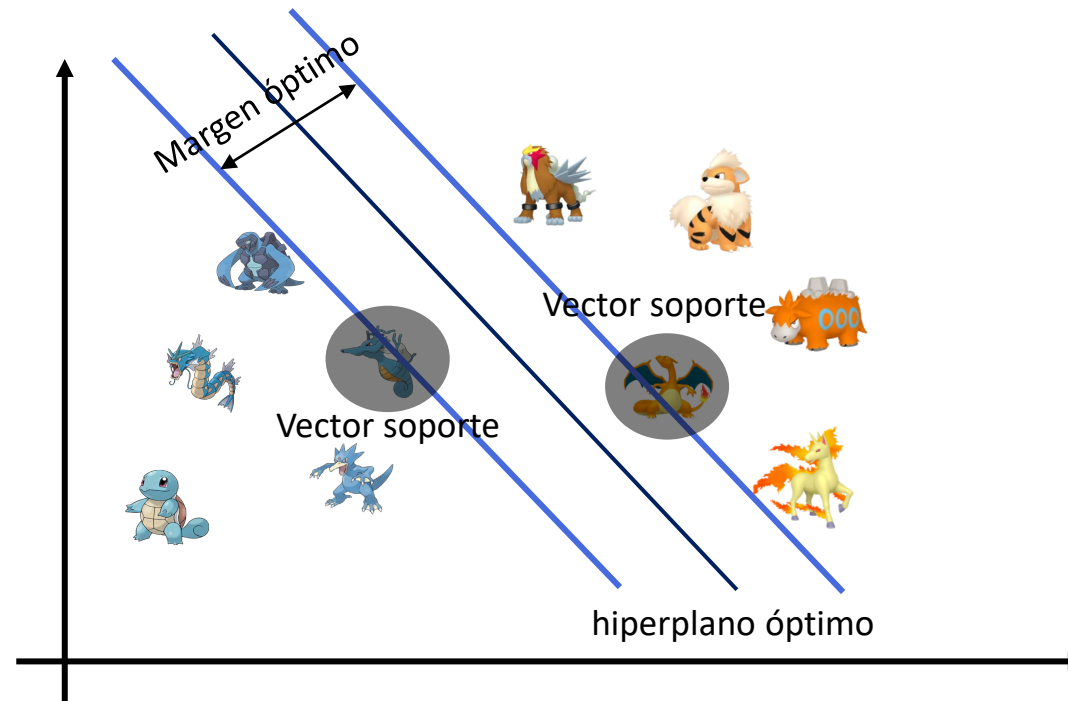
Dado que estamos representando los puntos de datos en un gráfico bidimensional, llamamos a este límite de decisión una línea recta, pero si tenemos más dimensiones, lo llamamos "hiperplano".



SVM: Hiperplano separador

El mejor hiperplano es aquel que tiene la máxima distancia entre ambas clases, y este es el objetivo principal de SVM.

Esto se logra buscando diferentes hiperplanos que clasifiquen las etiquetas de la mejor manera, y luego se elige el que esté más alejado de los puntos de datos o el que tenga el margen máximo.



Intuición matemática

Comprensión del producto escalar

Un vector es una cantidad con magnitud y dirección, y al igual que los números, se pueden usar operaciones matemáticas como la suma y la multiplicación.

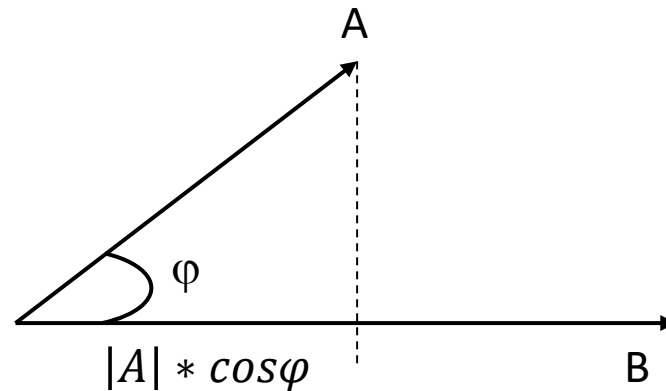
La multiplicación de vectores, que puede realizarse de dos maneras: producto escalar y producto vectorial

La única diferencia radica en que el producto escalar se utiliza para obtener un valor escalar como resultante, mientras que el producto vectorial se utiliza para obtener un vector.

El producto escalar se puede definir como la proyección de un vector sobre otro, multiplicado por el producto de otro vector.

Intuición matemática

Para hallar el producto escalar entre ellos, primero calculamos la magnitud de ambos vectores (A y B) y, para ello, utilizamos el teorema de Pitágoras o la fórmula de la distancia. Después de hallar la magnitud, simplemente la multiplicamos por el ángulo coseno entre ambos vectores.



$$A \cdot B = |A| * \cos \varphi * |B|$$

$|A| * \cos \varphi$ es la proyección de A sobre B

$|B|$ es la magnitud del vector B

En SVM solo necesitamos la proyección de A, no la magnitud de B.

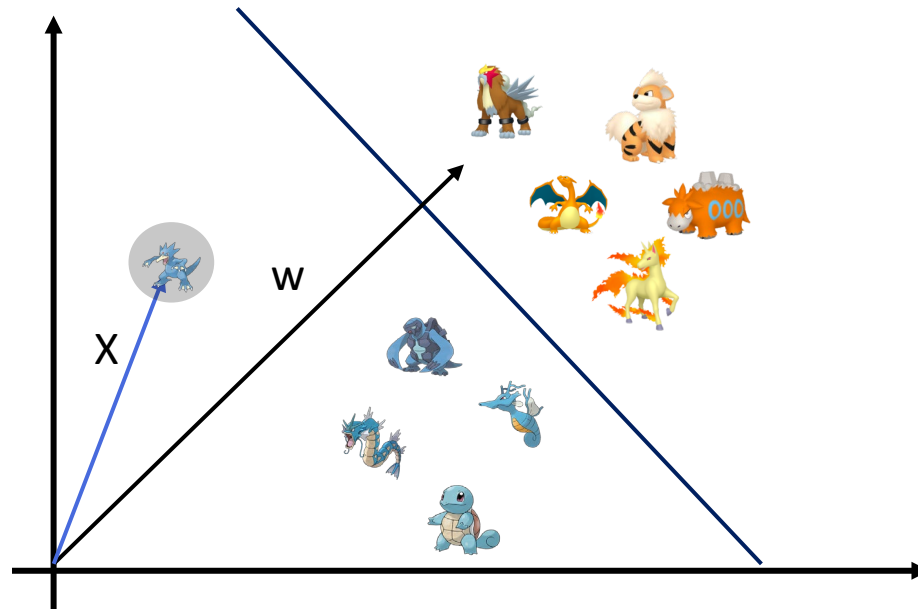
Para obtener la proyección, simplemente se puede tomar el vector unitario de B, ya que estará en la dirección de B, pero su magnitud será 1.

$$A \cdot B = |A| * \cos \varphi * \text{vector unitario de B}$$

Intuición matemática

Consideremos un punto aleatorio X y queremos saber si se encuentra en el lado derecho o izquierdo del plano (positivo o negativo).

Primero asumimos que este punto es un vector (X) y luego creamos un vector (w) perpendicular al hiperplano.

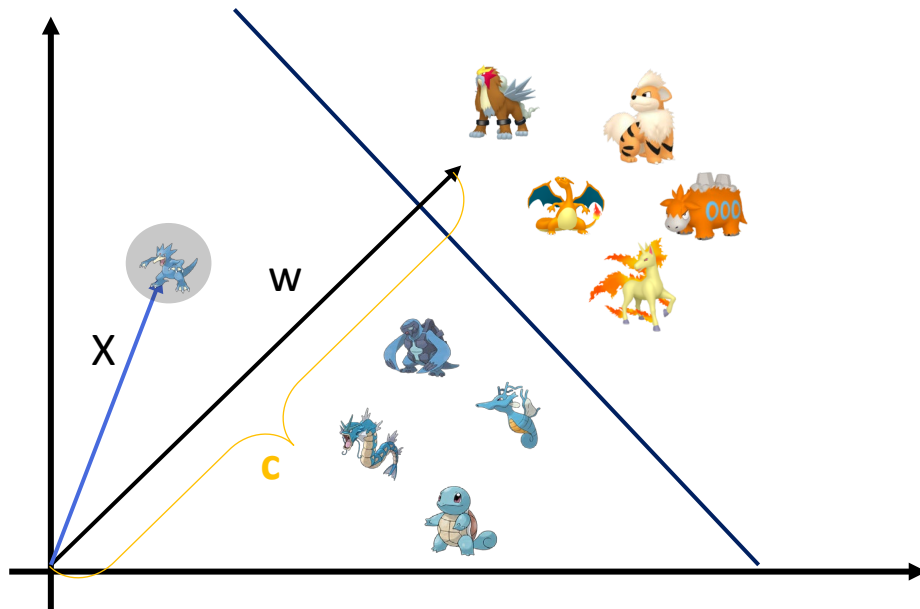


Intuición matemática

Supongamos que la distancia del vector w desde el origen hasta el límite de decisión es « c ».

La proyección de cualquier vector se denomina producto escalar. Por lo tanto, calculamos el producto escalar de los vectores x y w .

- Si el producto escalar es mayor que « c », podemos decir que el punto se encuentra en el lado derecho.
- Si el producto escalar es menor que « c », el punto se encuentra en el lado izquierdo.
- Si es igual a « c », el punto se encuentra en la frontera de decisión.



$\vec{X} \cdot \vec{w} = c$ (el punto se encuentra en la frontera de decisión)

$\vec{X} \cdot \vec{w} > c$ (ejemplos positivos)

$\vec{X} \cdot \vec{w} < c$ (ejemplos negativos)

SVM: Vector perpendicular (w)

Una duda frecuente es por qué se elige el vector perpendicular w al hiperplano.

Como el objetivo es medir la distancia de un vector X desde el límite de decisión y dado que hay infinitos puntos en el límite, medir la distancia desde todos ellos resulta impráctico.

Para estandarizar, se utiliza el vector perpendicular w como referencia y por tanto se proyectan todos los demás puntos de datos sobre este vector perpendicular y comparamos sus distancias.

SVM: Definición de margen

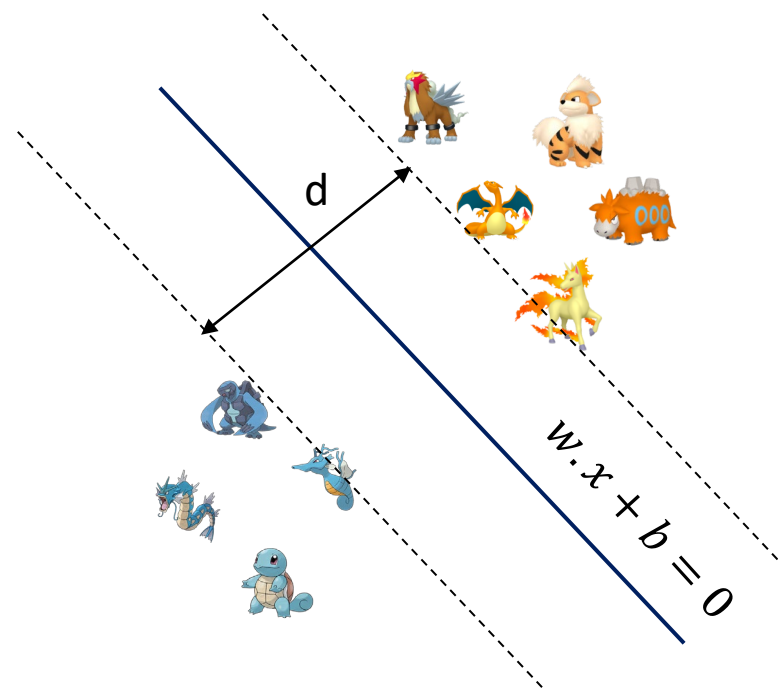
La ecuación de un hiperplano es $w \cdot x + b = 0$ con w un vector normal al hiperplano y b un desplazamiento.

Para clasificar un punto como fuego (+) o agua (-), necesitamos definir una regla de decisión.

$$y = \begin{cases} +1 & \text{si } \vec{X} \cdot \vec{w} + b \geq 0 \\ -1 & \text{si } \vec{X} \cdot \vec{w} + b < 0 \end{cases}$$

Si el valor $w \cdot x + b > 0$, decimos que es fuego (+); de lo contrario, es agua (-).

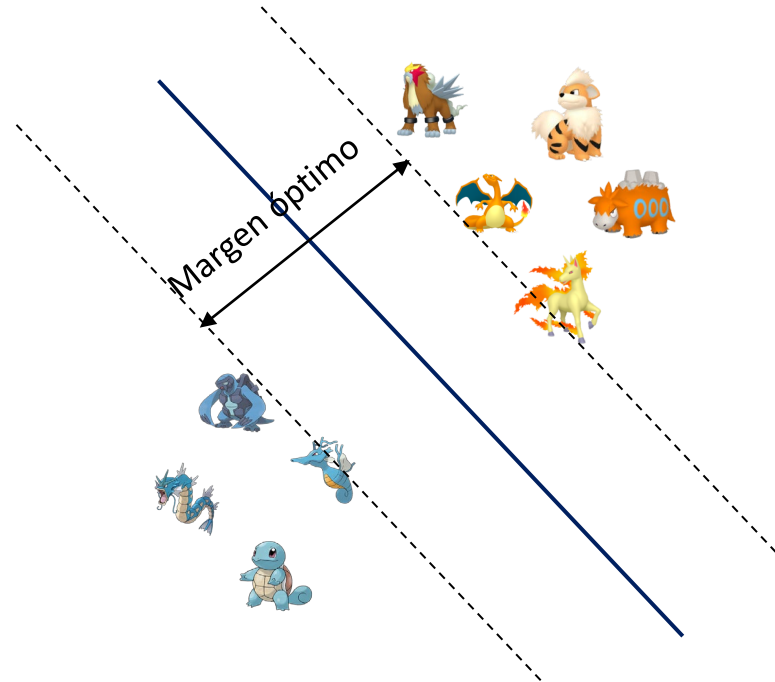
Ahora se necesita encontrar el par (w, b) tal que el margen tenga una distancia máxima.



SVM: Definición de margen

Como las líneas paralelas dependen de (w, b) en el hiperplano. Si multiplicamos la ecuación del hiperplano por un factor mayor que 1, las líneas paralelas se contraerán, y si la multiplicamos por un factor menor que 1, se expandirán.

El objetivo de las SVM es encontrar el hiperplano separador óptimo que maximice el margen de los datos de entrenamiento, es decir, la distancia (d). Sin embargo, existen algunas restricciones para esta distancia (d).



SVM: Función de optimización y sus restricciones

Para obtener la función de optimización, se deben considerar algunas restricciones.

Esta restricción es calcular la distancia (d) de tal manera que ningún punto, fuego o agua, pueda cruzar la línea de margen.

$$\vec{X} \cdot \vec{w} + b \leq -1 \text{ (ejemplos negativos – agua)}$$

$$\vec{X} \cdot \vec{w} + b \geq 1 \text{ (ejemplos positivos – fuego)}$$

En lugar de considerar dos restricciones, intentaremos simplificarlas en una sola.

Suponemos que las clases negativas (agua) tienen $y = -1$ y las positivas (fuego) $y = 1$.

Podemos afirmar que, para que cada punto se clasifique correctamente, esta condición siempre debe cumplirse:

$$y_i(\vec{X} \cdot \vec{w} + b) \geq 1$$

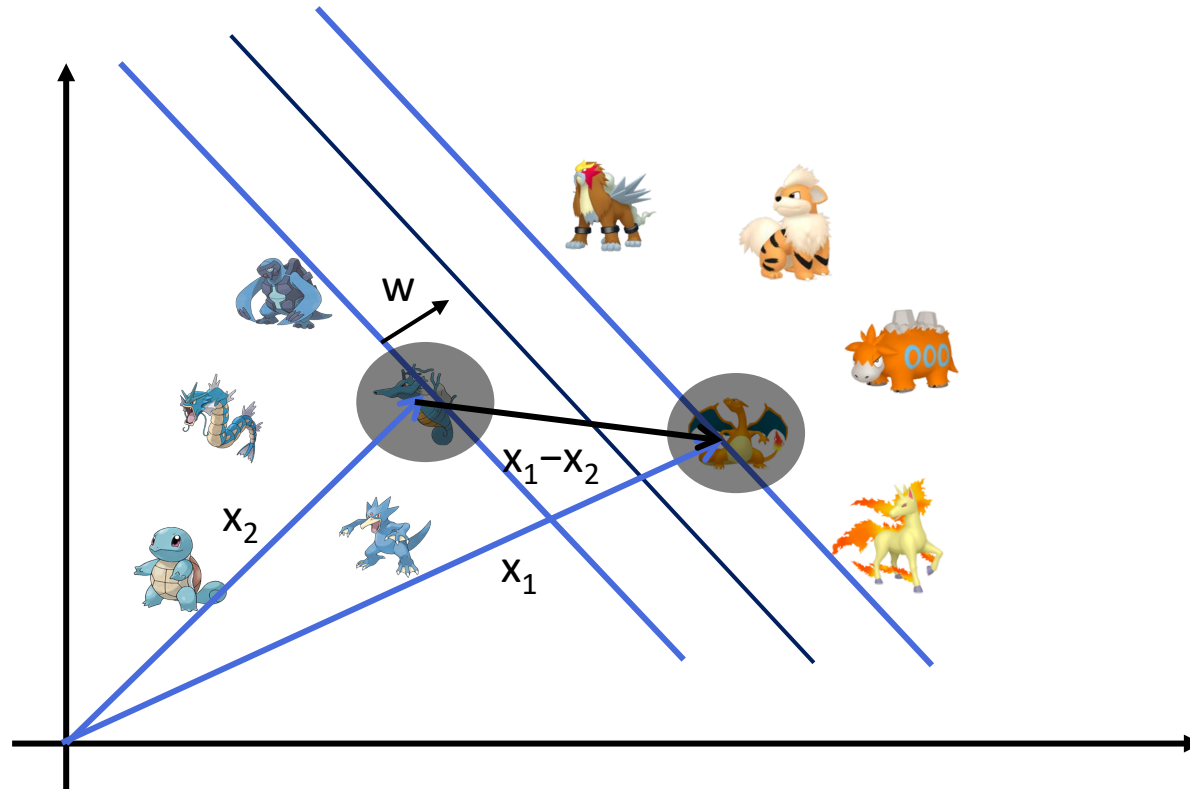
SVM: Función de optimización y sus restricciones

Nuestro objetivo es encontrar la distancia más corta entre estos dos puntos.

Esto se puede lograr usando un truco del producto escalar:

1. Tomar un vector w perpendicular al hiperplano.
2. Hallar la proyección del vector $(x_1 - x_2)$ sobre w mediante el producto escalar de ambos vectores:

$$\frac{x_1 \cdot \vec{w} - x_2 \cdot \vec{w}}{\|w\|}$$



SVM: Cálculo del margen y del hiperplano separador óptimo

Como x_2 y x_1 son vectores de soporte y se encuentran en el hiperplano, por lo tanto seguirán la ecuación de la recta $y_i(2x + b) = 1$.

Para ejemplos positivos (fuego): $\vec{w} \cdot x_2 = 1 - b$

Para ejemplos negativos (agua): $\vec{w} \cdot x_1 = -1 - b$

Ahora reemplazamos estas dos ecuaciones en:

$$\frac{x_2 \cdot \vec{w} - x_1 \cdot \vec{w}}{\|w\|} = \frac{(1 - b) - (-1 - b)}{\|w\|} = \frac{2}{\|w\|}$$

Por lo tanto la ecuación que tenemos que maximizar es: $\operatorname{argmax}(w^*, b^*) \frac{2}{\|w\|}$ tal que $y_i(\vec{X} \cdot \vec{w} + b) \geq 1$

Hemos encontrado nuestra función de optimización, pero existe un inconveniente: no encontramos este tipo de datos perfectamente separables linealmente en la industria.

Este problema se denomina SVM de *margen duro*.

SVM: Cálculo del margen y del hiperplano separador óptimo

El proceso de optimización se completa con el método de los multiplicadores de Lagrange, adecuado para encontrar máximos y mínimos de funciones continuas de múltiples variables y sujetas a restricciones.

En dos dimensiones, los multiplicadores de Lagrange permiten resolver el problema de maximizar una función $f(x, y)$ sujeta a las restricciones $g(x, y) = 0$, con la única condición de que ambas tengan derivadas parciales continuas, es decir, que su forma sea «suave» en todas sus variables.

De este modo, Lagrange optimizará la función $f(x, y) - \lambda \cdot g(x, y)$, donde λ es el coste de la restricción.

Esto es generalizable a cualquier número de dimensiones.

SVM: Margen blando

Para abordar este problema, modificamos la ecuación de forma que permita pocas clasificaciones erróneas, es decir, pocos puntos se clasifiquen incorrectamente.

El máximo de una función ($\max[f(x)]$) también puede escribirse como $\min[1/f(x)]$.

Es habitual minimizar una función de coste en problemas de optimización; por lo tanto, podemos invertir la función siempre que esta función sea invertible.

$$\operatorname{argmin}(w^*, b^*) \frac{\|w\|}{2} \text{ tal que } y_i(\vec{X} \cdot \vec{w} + b) \geq 1$$

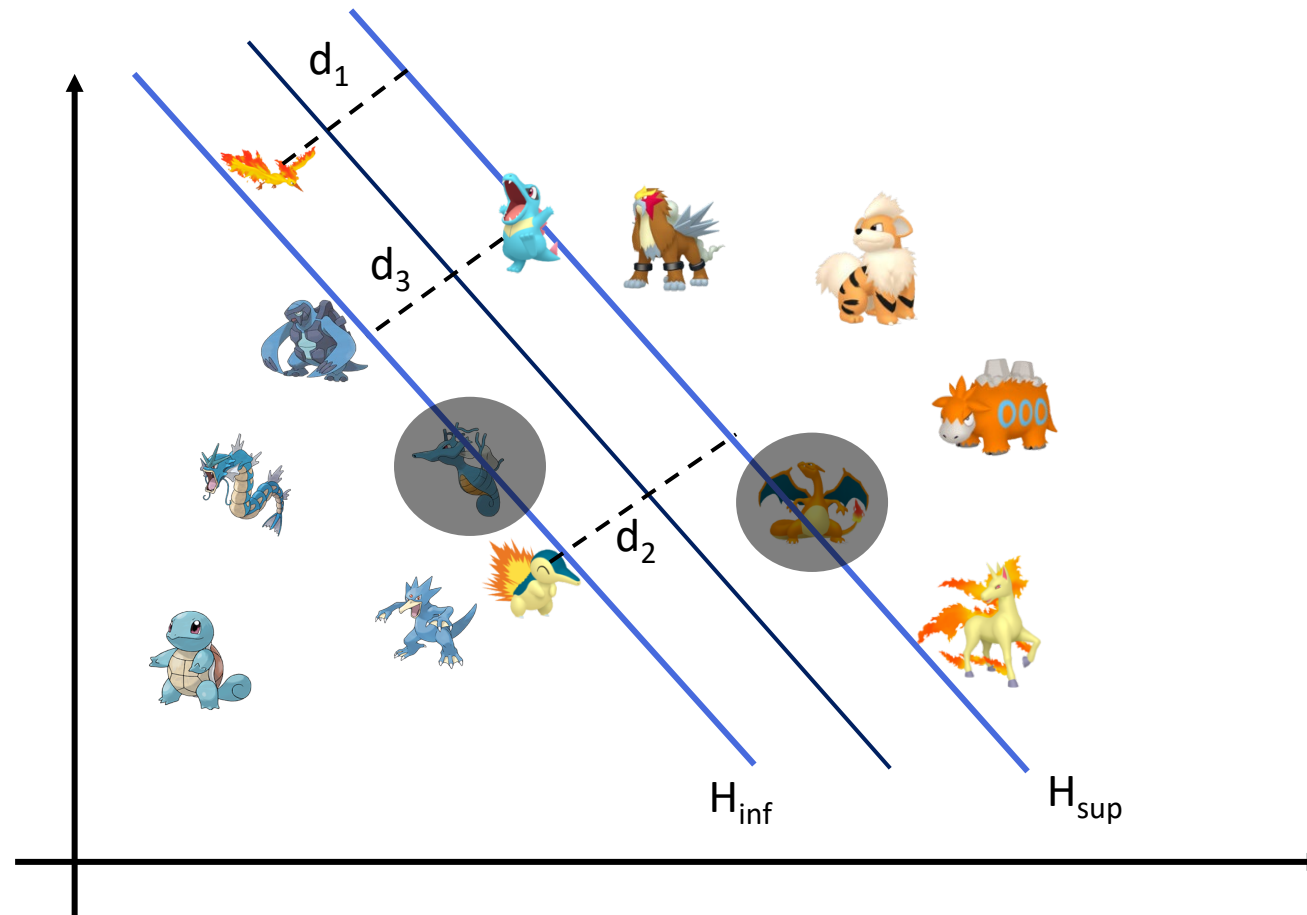
Para hacer una ecuación de *margen blando* se agrega zeta (ζ) y un hiperparámetro 'c'

$$\operatorname{argmin}(w^*, b^*) \frac{\|w\|}{2} + c \sum_{i=1}^n \zeta_i$$

Para todos los puntos clasificados correctamente, zeta será igual a 0 y para todos los puntos clasificados incorrectamente, zeta es simplemente la distancia de ese punto particular hasta su hiperplano correcto.

SVM: Margen blando

Si vemos los puntos positivos (fuego) clasificados incorrectamente, el valor de zeta será la distancia de estos puntos desde el hiperplano H_{sup} y para los puntos negativos (agua) clasificados incorrectamente, el zeta será la distancia de ese punto desde el hiperplano H_{inf} .

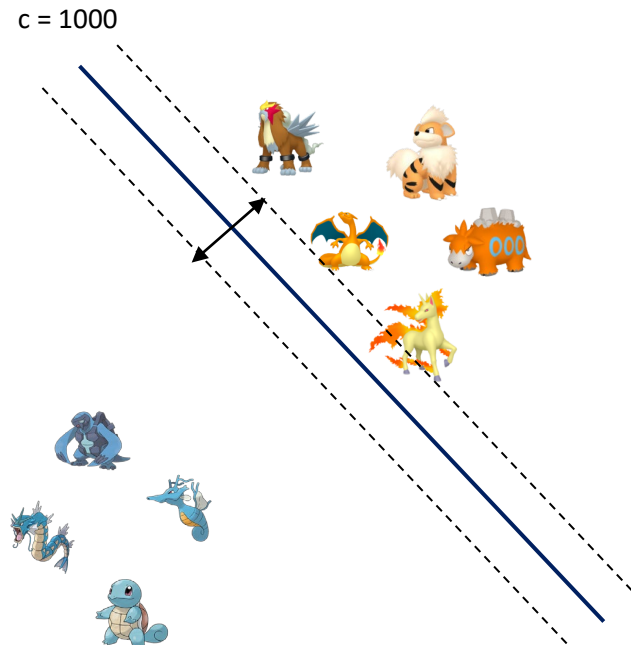


SVM: Margen blando

El Error del SVM es igual al Error del Margen + Error de Clasificación.

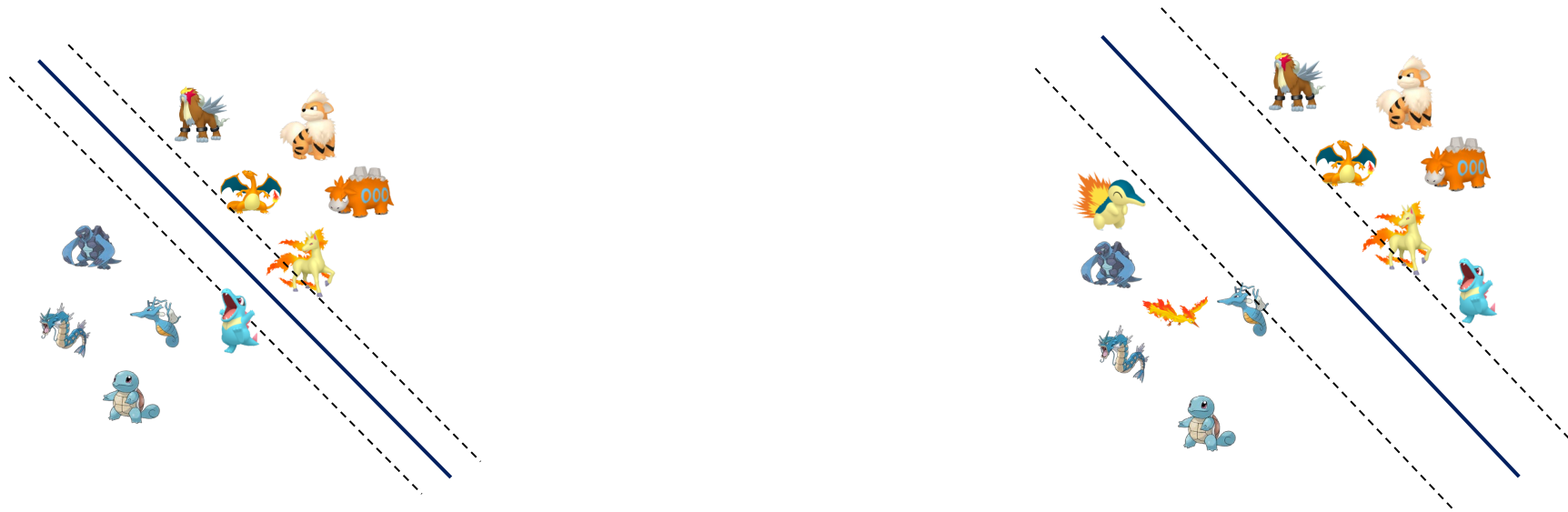
Cuanto mayor sea el margen, menor será el posible error de margen, y viceversa.

Si tomamos un valor alto de " c " = 1000; significa que no se desea centrarse en el error de margen y solo se busca un modelo que no clasifique erróneamente ningún dato.



SVM: Margen blando vs Margen duro

Ahora, *cuál es el mejor modelo*: ¿el que tiene el margen máximo y dos puntos mal clasificados, o el que tiene un margen mínimo y todos los puntos están correctamente clasificados?



Si no se desea ninguna clasificación errónea en el modelo, la elección es la figura de la izquierda. Esto significa que aumentaremos "c" para disminuir el Error de Clasificación.

Si queremos maximizar el margen la elección es la figura de la derecha, el valor de "c" debe minimizarse y se debería encontrar su valor óptimo usando GridsearchCV y validación cruzada.

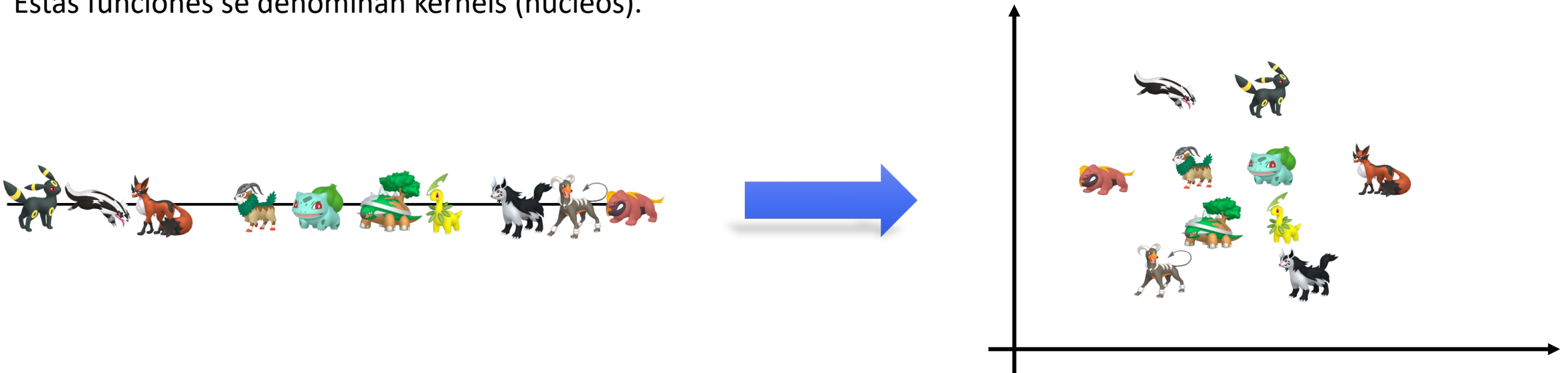
SVM: Kernels

La característica más interesante de SVM es que puede trabajar con conjuntos de datos no lineales mediante el uso del “Truco del Kernel” que facilita su clasificación.

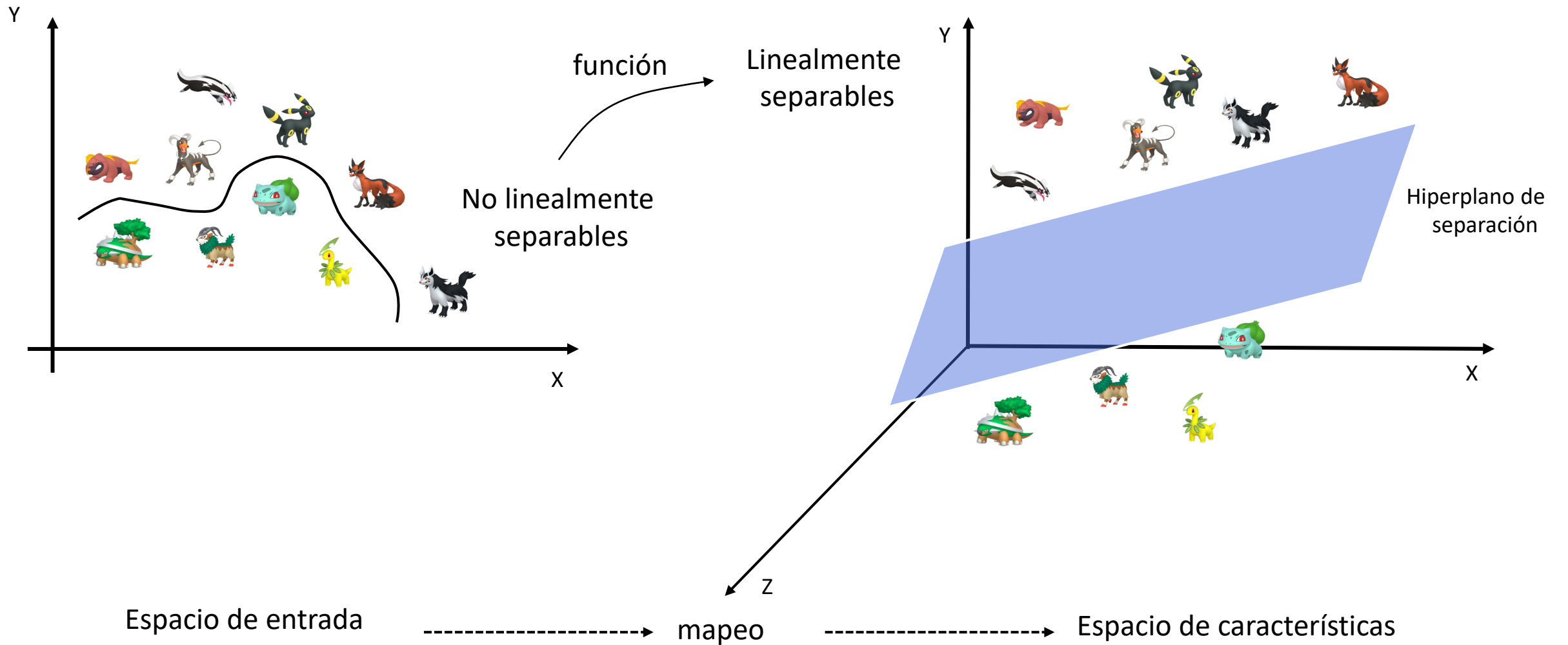
Si tenemos un conjunto de datos con la figura de la izquierda, aquí vemos que no podemos trazar una sola línea, o hiperplano, para clasificar correctamente los puntos.

El “Truco del Kernel” convierte este espacio de menor dimensión a uno de mayor dimensión mediante funciones cuadráticas que nos permiten encontrar un límite de decisión que los separe.

Estas funciones se denominan kernels (núcleos).



SVM: Kernels



SVM: Tipos de kernels

Una de las tareas que realiza el algoritmo SVM es identificar los vectores de soporte, i.e., aquellos vectores que determinan el hiperplano separador.

Kernel lineal

$$K(u, v) = u^T \cdot v$$

Kernel polinomial

$$K(u, v) = (\gamma u^T \cdot v + b)^p$$

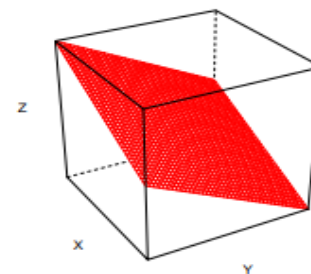
Kernel radial

$$K(u, v) = e^{-\gamma \|u - v\|^2}$$

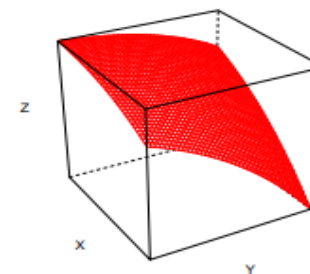
Kernel sigmoidal

$$K(u, v) = \tanh(\gamma u^T \cdot v + b)$$

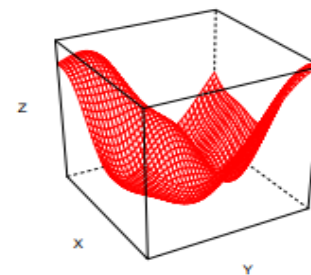
donde $u, v \in X$ y $\gamma > 0$, $b \geq 0$ y $p > 0$ son parámetros



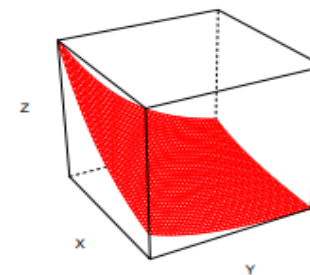
(a) Kernel lineal



(b) Kernel polinomial



(c) Kernel radial



(d) Kernel sigmoidal

SVM: Elección del Kernel

La selección del kernel depende del tipo de conjunto de datos.

Para datos linealmente separables, usa un kernel lineal. Es simple y tiene menor complejidad que otros kernels.

Comienza asumiendo que tus datos son linealmente separables y prueba primero con el kernel lineal.

Si es necesario, pasa a kernels más complejo como RBF.

Los kernels polinómicos se usan raramente debido a su baja eficiencia.

Si los kernels lineal y RBF dan resultados similares, es recomendable elegir la opción más simple, que es el kernel lineal.

Algoritmo SVM

Entradas:

1. Un conjunto de datos $X = \{x_1, x_2, \dots, x_n\}$ en un espacio de alta dimensión $\mathbb{R}^d$
2. Un vector $Y = \{y_1, y_2, \dots, y_n\}$ con las etiquetas de las clases
3. Parámetros: Factor de regularización (C), Tipo de kernel (K) y sus parámetros específicos

Pasos del algoritmo:

1. Calcular matriz de kernel K
2. Inicializar multiplicadores de Lagrange $\alpha_i = 0$
3. Verificar la convergencia.
 - Sin convergencia: Seleccionar par de multiplicadores (α_i, α_j) , Optimizar α_i y α_j , Actualizar α_i y α_j , Calcular término de sesgo.
4. Identificar vectores de soporte ($\alpha_i > 0$)
5. Calcular $w = \sum \alpha_i * y_i * x_i$ (para kernel lineal)
6. Guardar modelo (vectores de soporte, α , b, parámetros del kernel)

Salida:

1. Vectores de soporte: Los puntos que definen el margen
2. Coeficientes del hiperplano: Los pesos que definen el hiperplano separador
3. Intercepto: El término independiente del hiperplano
4. Función de decisión: Permite clasificar nuevos puntos

Algoritmo SVM

Partimos de una clasificación binaria, i.e., donde cada punto de nuestro espacio de características, $x_i \in X$, estará asociado a una clase binaria $\{-1, 1\}$ para entender el concepto.

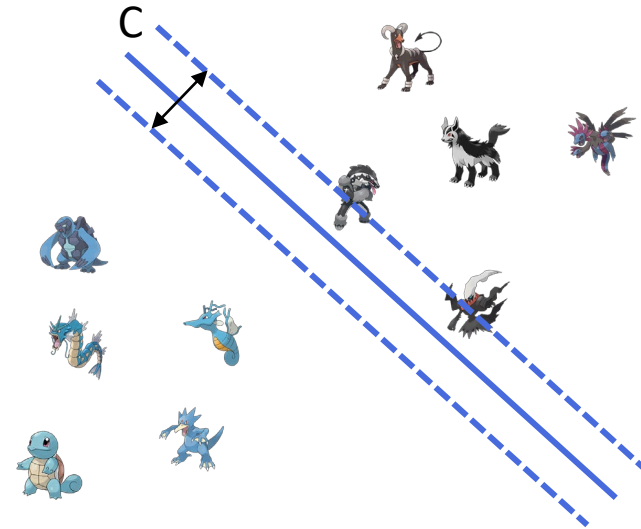
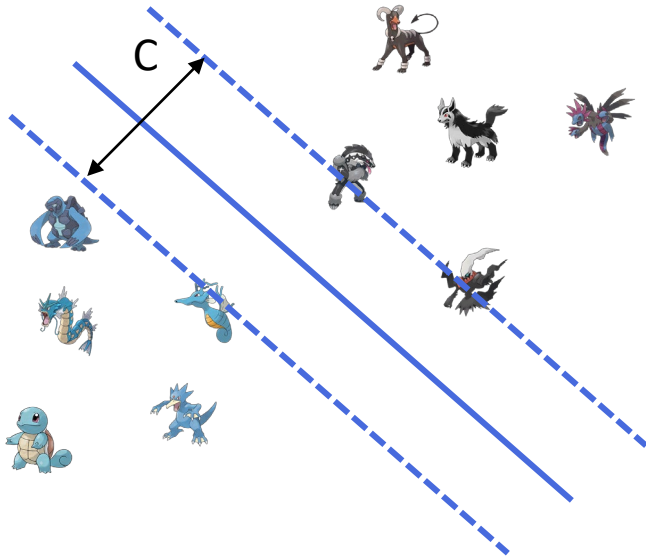
En este caso, nuestra función de clasificación podría consistir en calcular la suma de distancias alteradas por un peso w .

$$h(z) = \text{signo} \left(\sum_{i=1}^n w_i \cdot c_i \cdot k(x_i, z) \right)$$

- n es el número de entradas del conjunto de datos de entrenamiento.
- z es el nuevo punto a clasificar.
- $h(z) \in \{-1, 1\}$ es la función de clasificación.
- $k: X \cdot X \rightarrow R$ es la función kernel que mide la similitud entre puntos (producto escalar, distancia, etc.).
- El conjunto de pares $\{(x_i, c_i)\}_{i=1}^n$ son los puntos etiquetados del juego de datos, i.e., constituyen el conjunto de datos de entrenamiento, donde $c_i \in \{-1, 1\}$ es la etiqueta o clase del punto x_i .
- $w_i \in R$ son los pesos que el algoritmo ha determinado para el conjunto de datos de entrenamiento.
- $\text{signo}()$ es la función que nos devuelve el signo de un número $\{+,-\}$.

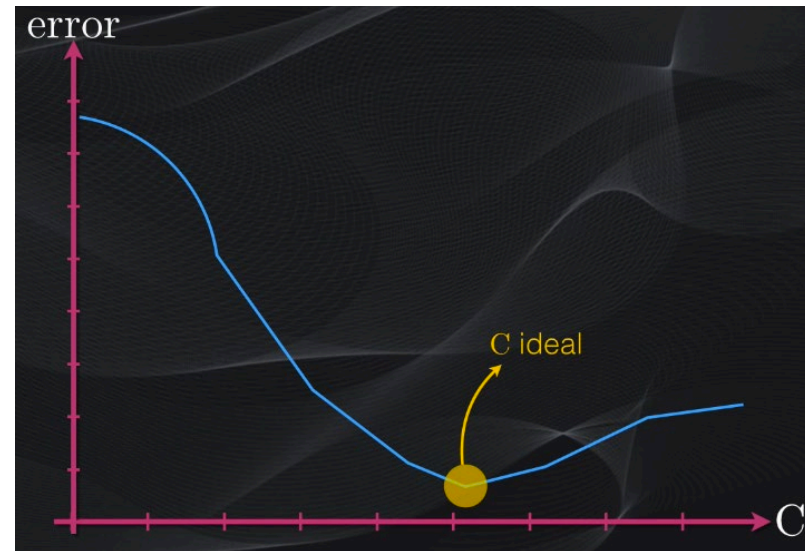
SVM: El parámetro C

Un C relativamente pequeño (menor que 1) permite generar márgenes amplios, y a medida que su tamaño aumenta el margen se va reduciendo poco a poco (mayor que 1).



SVM: Costo Computacional y el parámetro C

Este parámetro se escoge de manera empírica analizando el error que se obtiene en la clasificación vs. diferentes valores de C, y a este algoritmo de máquina de vectores de soporte se le conoce como “soft margin”.



Costo computacional

Debido a la utilización de kernels, en el ajuste de un SVM participa una matriz $n \times n$, donde n es el número de observaciones de entrenamiento. Por esta razón, lo que más influye en el tiempo de computación necesario para entrenar un SVM es el número de observaciones, no el de predictores.

SVM: Clasificación multi-clase

El concepto de hiperplano de separación en el que se basan los SVMs no se generaliza de forma natural para más de dos clases. Se han desarrollado numerosas estrategias con el fin de aplicar este algoritmo a problemas multiclase, de entre ellos, los más empleados son: one-versus-one, one-versus-all y DAGSVM

One-versus-all

Esta estrategia consiste en ajustar K SVMs distintos, cada uno comparando una de las n clases frente a las restantes $n-1$ clases. Como resultado, se obtiene un hiperplano de clasificación para cada clase.

Para obtener una predicción, se emplean cada uno de los K clasificadores y se asigna la observación a la clase para la que la predicción resulte positiva.

Esta aproximación, aunque sencilla, puede causar inconsistencias, ya que puede ocurrir que más de un clasificador resulte positivo, asignando así una misma observación a diferentes clases.

Otro inconveniente adicional es que cada clasificador se entrena de forma no balanceada.

Por ejemplo, si el set de datos contiene 100 clases con 10 observaciones por clase, cada clasificador se ajusta con 10 observaciones positivas y 990 negativas.

SVM: Clasificación multi-clase

One-versus-one (scikit-learn en Python)

Se entrena un clasificador SVM binario para cada par posible de clases. Para un problema con n clases, se crean $n(n-1)/2$ clasificadores.

Proceso de entrenamiento:

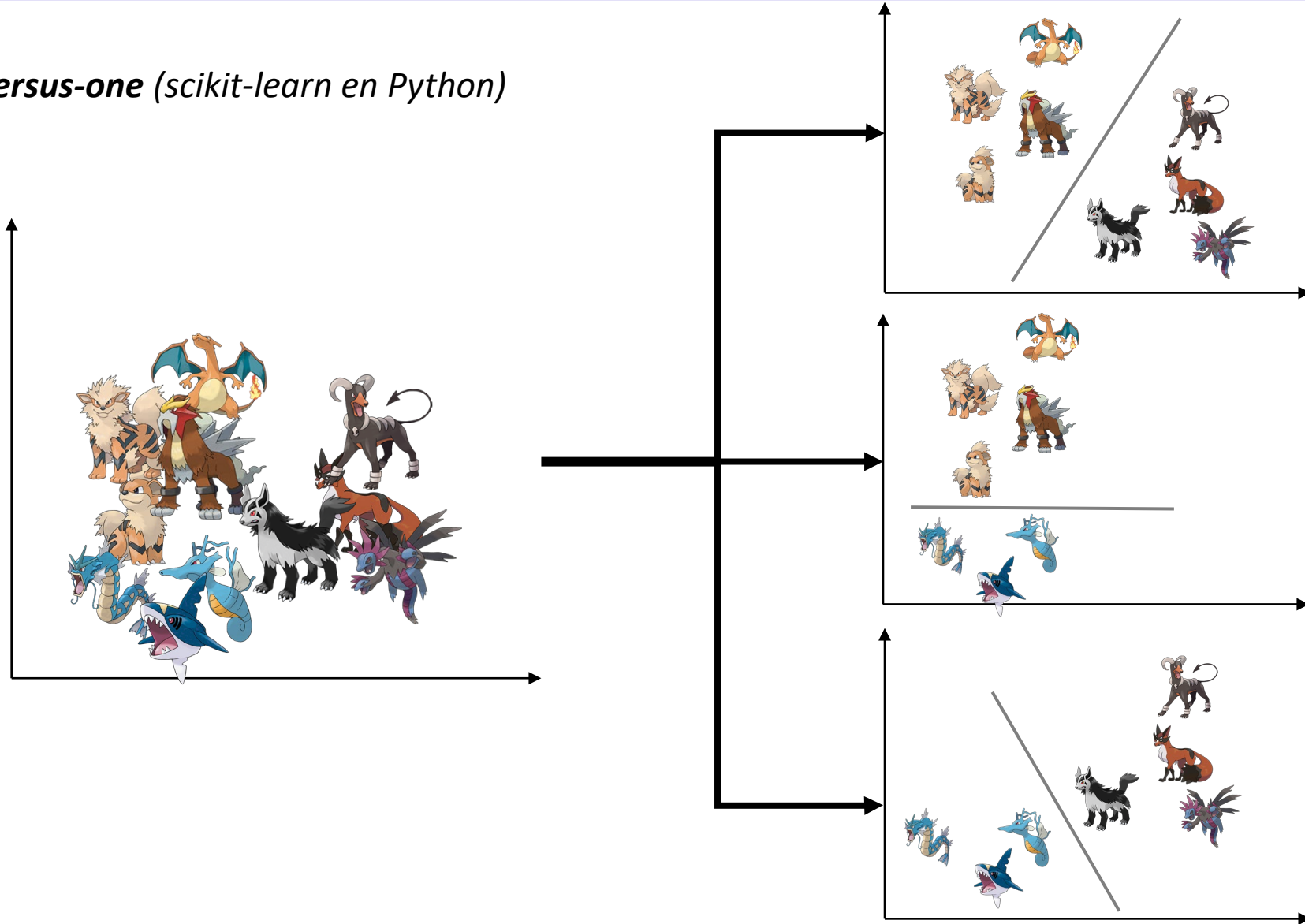
1. Para cada par de clases (i, j) , se entrena un SVM usando solo los datos de esas dos clases.
2. El SVM aprende a distinguir entre la clase i y la clase j .
3. Este proceso se repite para todos los pares posibles de clases.

Clasificación de nuevos datos:

1. Cuando se presenta un nuevo dato, se ejecuta a través de todos los clasificadores binarios.
2. Cada clasificador "vota" por una de las dos clases que fue entrenado para distinguir.
3. La clase que recibe más votos en total es la predicción final.

SVM: Clasificación multi-clase

One-versus-one (scikit-learn en Python)



SVM: Clasificación multi-clase

One-versus-one (*scikit-learn en Python*)

Ventajas:

- Mejor manejo de clases desbalanceadas. Cada clasificador se entrena con subconjuntos más pequeños y balanceados.
- Puede ser más preciso en problemas complejos con muchas clases.
- Los clasificadores individuales son más simples y rápidos de entrenar.

Desventajas:

- Requiere entrenar y almacenar más clasificadores que One vs All.
- El proceso de votación puede ser computacionalmente costoso para la clasificación.
- Posibilidad de empates en la votación, que requieren estrategias de desempate.

Ventajas de SVM

- Funciona bien con datos complejos: SVM es ideal para conjuntos de datos donde la separación entre categorías no es clara. Puede gestionar eficazmente datos lineales y no lineales.
- Eficaz en espacios de alta dimensión: SVM funciona bien incluso con más características (dimensiones) que muestras, lo que lo hace útil para tareas como la clasificación de texto o el reconocimiento de imágenes.
- Evita el sobreajuste: SVM se centra en encontrar el mejor límite de decisión (margen) entre clases, lo que ayuda a reducir el riesgo de sobreajuste, especialmente en datos de alta dimensión.
- Versátil con kernels: Al utilizar diferentes funciones kernel (como funciones de base lineales, polinómicas o radiales), SVM puede adaptarse a diversos tipos de datos y resolver problemas complejos.
- Robusto ante valores atípicos: SVM se ve menos afectado por los valores atípicos porque se centra en los vectores de soporte (puntos de datos más cercanos al margen), lo que ayuda a crear un modelo más generalizado.

Desventajas de SVM

- Lento con grandes conjuntos de datos: SVM puede ser computacionalmente costoso y lento de entrenar, especialmente cuando el conjunto de datos es muy grande.
- Difícil de ajustar: Elegir el kernel y los parámetros correctos (como C y γ) puede ser complicado y, a menudo, requiere mucho ensayo y error.
- No apto para datos ruidosos: Si el conjunto de datos tiene demasiadas clases superpuestas o ruido, SVM puede tener dificultades para funcionar correctamente porque intenta encontrar una separación perfecta.
- Difícil de interpretar: A diferencia de otros algoritmos, los modelos SVM no son fáciles de interpretar ni explicar, especialmente cuando se utilizan kernels no lineales.
- Consumen mucha memoria: SVM requiere almacenar los vectores de soporte, lo que puede ocupar mucha memoria, lo que lo hace menos eficiente para conjuntos de datos muy grandes.

Ejemplo en python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import style
import seaborn as sns
from mlxtend.plotting import plot_decision_regions
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn import metrics
```

```
plt.rcParams['image.cmap'] = "bwr"
plt.rcParams['savefig.bbox'] = "tight"
style.use('ggplot') or plt.style.use('ggplot')
```

```
import warnings
warnings.filterwarnings('ignore')
```

Ejemplo en python

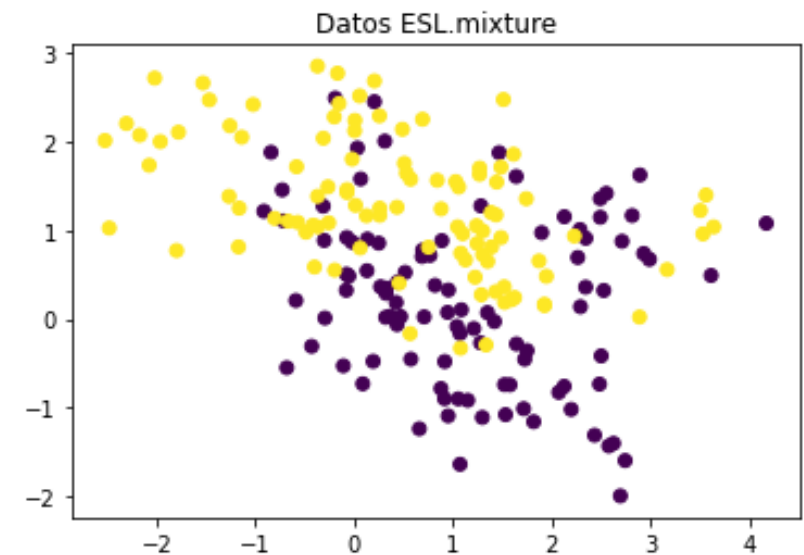
```
url = 'https://raw.githubusercontent.com/JoaquinAmatRodrigo/' \
      + 'Estadistica-machine-learning-python/master/data/ESL.mixture.csv'
datos = pd.read_csv(url)
datos.head(3)
```

```
fig, ax = plt.subplots(figsize=(6,4))
ax.scatter(datos.X1, datos.X2, c=datos.y);
ax.set_title("Datos ESL.mixture");
```

```
X = datos.drop(columns = 'y')
y = datos['y']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y.values.reshape(-1,1),
                                                    train_size = 0.8, random_state = 1234, shuffle = True)
```

```
modelo = SVC(C = 100, kernel = 'linear', random_state=123)
modelo.fit(X_train, y_train)
```



Ejemplo en python

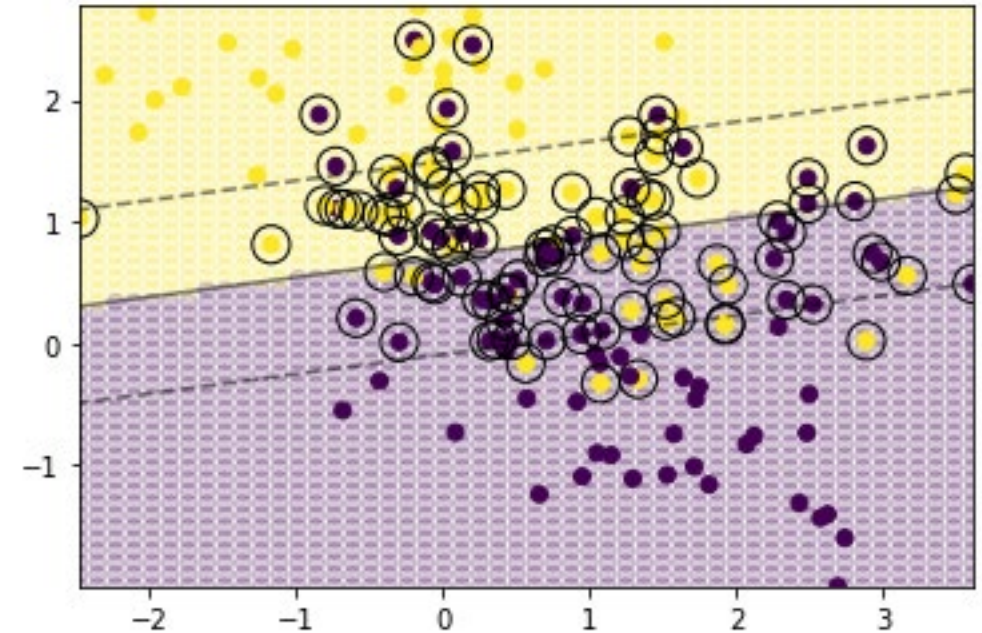
```
x = np.linspace(np.min(X_train.X1), np.max(X_train.X1), 50)
y = np.linspace(np.min(X_train.X2), np.max(X_train.X2), 50)
Y, X = np.meshgrid(y, x)
grid = np.vstack([X.ravel(), Y.ravel()]).T
```

```
pred_grid = modelo.predict(grid)
```

```
fig, ax = plt.subplots(figsize=(6,4))
ax.scatter(grid[:,0], grid[:,1], c=pred_grid, alpha = 0.2)
ax.scatter(X_train.X1, X_train.X2, c=y_train, alpha = 1)
```

```
ax.scatter(
    modelo.support_vectors_[:, 0],
    modelo.support_vectors_[:, 1],
    s=200, linewidth=1, facecolors='none', edgecolors='black')
```

```
ax.contour(X, Y, modelo.decision_function(grid).reshape(X.shape),
    colors = 'k', levels = [-1, 0, 1], alpha = 0.5, linestyles = ['--', '-', '--'])
```



Ejemplo en python

```
predicciones = modelo.predict(X_test)
predicciones

array([1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0,
       1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0], dtype=int64)

accuracy = accuracy_score(y_true = y_test, y_pred = predicciones,
                           normalize = True)
print("")
print(f"El accuracy de test es: {100*accuracy}%")

El accuracy de test es: 70.0%
```

Ejemplo en python

```
# SVM radial
param_grid = {'C': np.logspace(-5, 7, 20)}

grid = GridSearchCV(
    estimator = SVC(kernel= "rbf", gamma='scale'),
    param_grid = param_grid,
    scoring = 'accuracy', n_jobs = -1,
    cv = 3, verbose = 0,
    return_train_score = True
)

# Se asigna el resultado a _ para que no se imprima por pantalla
_ = grid.fit(X = X_train, y = y_train)

resultados = pd.DataFrame(grid.cv_results_)
resultados.filter(regex = '(param.*|mean_t|std_t)')\
    .drop(columns = 'params')\
    .sort_values('mean_test_score', ascending = False) \
    .head(5)
```


Ejemplo en python

	param_C	mean_test_score	...	mean_train_score	std_train_score
8	1.128838	0.762520	...	0.790778	0.035372
12	379.269019	0.750641	...	0.868777	0.007168
7	0.263665	0.750175	...	0.778228	0.026049
9	4.83293	0.744118	...	0.815729	0.026199
11	88.586679	0.738062	...	0.859431	0.019840

[5 rows x 5 columns]

```
print("Mejores hiperparámetros encontrados (cv)")  
print(grid.best_params_, ":", grid.best_score_, grid.scoring)
```

```
modelo = grid.best_estimator_
```

```
{'C': 1.1288378916846884} : 0.7625203820172374 accuracy
```

Ejemplo en python

```
x = np.linspace(np.min(X_train.X1), np.max(X_train.X1), 50)
y = np.linspace(np.min(X_train.X2), np.max(X_train.X2), 50)
Y, X = np.meshgrid(y, x)
grid = np.vstack([X.ravel(), Y.ravel()]).T
```

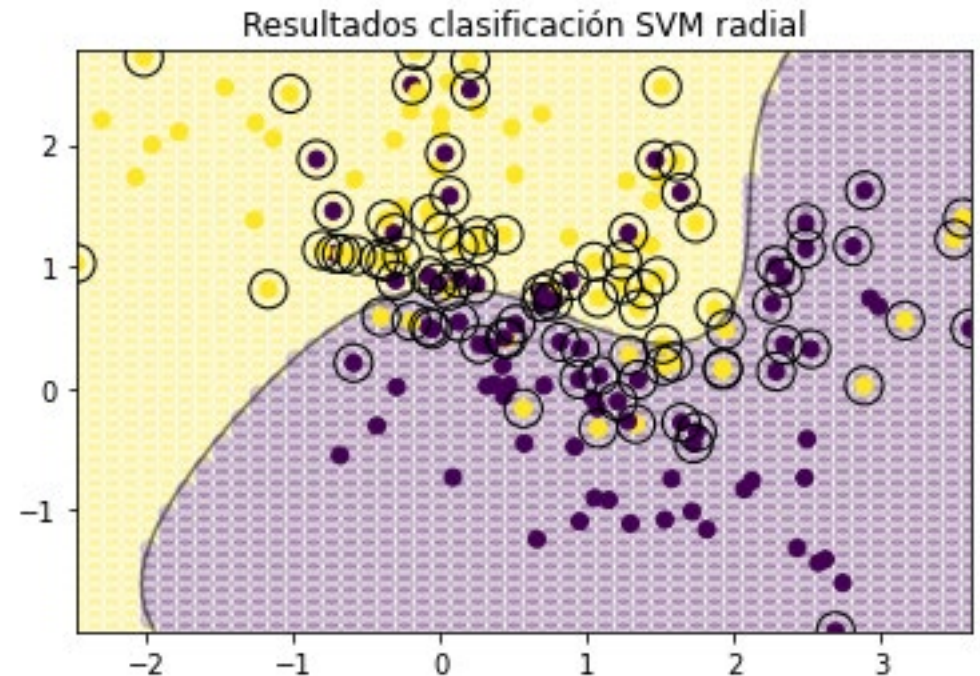
```
pred_grid = modelo.predict(grid)
```

```
fig, ax = plt.subplots(figsize=(6,4))
ax.scatter(grid[:,0], grid[:,1], c=pred_grid, alpha = 0.2)
ax.scatter(X_train.X1, X_train.X2, c=y_train, alpha = 1)
```

```
ax.scatter(
    modelo.support_vectors_[:, 0],
    modelo.support_vectors_[:, 1],
    s=200, linewidth=1, facecolors='none', edgecolors='black')
```

```
ax.contour(X, Y,
    modelo.decision_function(grid).reshape(X.shape),
    colors='k', levels=[0], alpha=0.5, linestyles='-')
```

```
ax.set_title("Resultados clasificación SVM radial");
```



Ejemplo en python

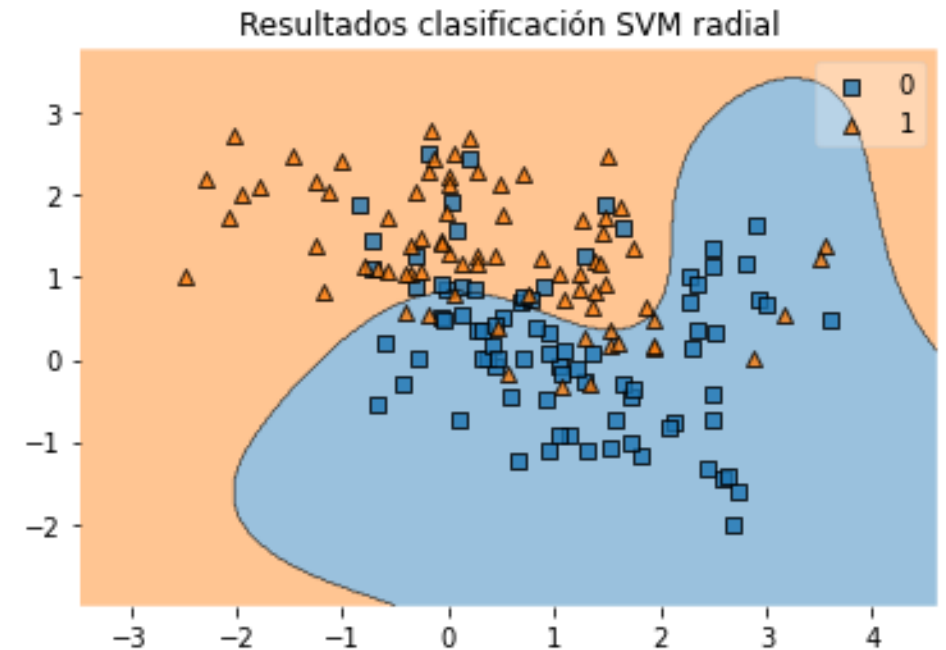
```
fig, ax = plt.subplots(figsize=(6,4))
plot_decision_regions(
    X = X_train.to_numpy(),
    y = y_train.flatten(), clf = modelo, ax = ax)

ax.set_title("Resultados clasificación SVM radial");

predicciones = modelo.predict(X_test)

accuracy = accuracy_score(
    y_true = y_test,
    y_pred = predicciones,
    normalize = True
)
print("")
print(f"El accuracy de test es: {100*accuracy}%")
```

El accuracy de test es: 80.0%



Ejemplo en python

```
confusion_matrix = pd.crosstab(  
    y_test.ravel(),  
    predicciones,  
    rownames=['Real'],  
    colnames=['Predicción']  
)
```

confusion_matrix

	Predicción	0	1
Real			
0		14	3
1		5	18